11

The Future Ahead

In this book, we started with how a neural network could digest text. As we have seen, neural networks do not do this natively but require the text to be processed. Simple neural networks can be used for some basic tasks such as classification, but human language carries an enormous amount of complex information.

In *Chapters 2* and *3*, we saw how we need sophisticated models in order to use semantic and syntactic information. The emergence of transformers and LLMs has made it possible to have models capable of reasoning and storing enormous amounts of factual knowledge. These multipurpose knowledge and skills have enabled LLMs to solve tasks for which they have not been trained (coding, solving math problems, and so on). Nevertheless, LLMs have problems such as a lack of specialized domain knowledge, continual learning, being able to use tools, and so on. Thus, from ***Chapter 4*** onward, we described systems that extend the capabilities of LLMs and which are designed to solve LLMs' problems.

In this chapter, we will discuss how some problems remain to be solved and what lies ahead in the future. We will start by presenting how agents can be used in different industries and the revolution that awaits us thanks to agents. Then, we will discuss some of the most pressing questions both technically and ethically.

In this chapter, we'll be covering the following topics:

- AI agents in healthcare
- AI agents in other sectors

- Challenges and open questions

AI agents in healthcare

One of the most exciting prospects for AI development is the possibility of having autonomous systems capable of conducting scientific discoveries on their own. This new paradigm is referred to as the *AI scientist*. Throughout this book, we have seen some examples of systems that are thought to be in accordance with this idea (ChemCrow, the virtual lab, and so on). In this section, we will discuss this paradigm in more detail: where the research is heading, the challenges faced, and future developments.

The idea behind an AI agent is to exploit LLMs in combination with tools (agents), as we have seen so far. In the future, researchers would like to add an experimental platform (an autonomous system able to conduct experiments by itself) to these systems so that they can conduct experiments independently. The complexity of biology could then be approached in a series of actionable tasks, where an LLM could break down a problem into a series of subtasks and autonomously solve them. The goal then would be to achieve discoveries not only more quickly but also more efficiently. The AI scientist would then be able to produce research at a speed and scale that would otherwise be impossible for humans.

In the first phase, humans would be at the center of the project. Scientists would provide input and criticism to the LLMs, and the models would incorporate this feedback into the process. During this iterative process, the model would analyze the problem, search the internet for information, and devise a plan, under human supervision (or otherwise using handcrafted prompts to guide it through the process). In such a scenario, an LLM would be an assistant to humans, where it proposes solutions and hypotheses. The ultimate goal would be to have an autonomous agent.

This vision is the culmination of a process that has been ongoing for decades in biomedical research. In fact, since the early 1990s, people have been talking about a new paradigm: the use of data-driven models. This paradigm shift has occurred because of technological advances and the

vast availability of data. Biomedical research produces a large amount of data, and in the last three decades, this information has begun to be centralized in a series of databases. Simultaneously with this integration and new accessibility of information, all sorts of tools have been developed by researchers. At first, these computational tools were models and statistical methods, but gradually, biomedical research has also benefited from machine learning and AI models. In a sense, the successes of one propelled the successes of the other, and vice versa. The more data was centralized and made available to the community, the more this allowed new models to be developed. Discoveries obtained through new models and methods prompted the production of new experiments and new data. For example, transcriptomics experiments allowed for large datasets, which were perfect for developing new machine learning models and tools. These models allowed some biological questions to be answered, and these answers led to new experiments and thus new data. AlphaFold2 was only possible because of the millions of structures on the **Protein Data Bank** (**PDB**). AlphaFold2 allowed researchers to produce new hypotheses, later confirmed by new experiments and new structures on the PDB. In addition, the limitations of AlphaFold2 led researchers to collect new data for specific questions. These new data and experimental verifications led to new models, creating positive feedback.

As you can see, when the LLMs arrived, fertile ground for further revolution was already present. First, a vast amount of data (millions of articles and huge databases of experimental data) was available, thus allowing models either to be trained on this data or to be able to search for information through dedicated databases. For example, a model could search for information it missed on biological sequences through dedicated APIs. Or, an LLM could use RAG to search for information on new articles. Second, the community produced thousands of models to solve specific tasks. An LLM does not need to know how to solve a task; there is a curated list of resources it can use for a whole range of subtasks. An LLM then does not need additional training but only needs to know how to orchestrate these specific task tools and models. At this point, we have everything we need to be able to create an agent system. Agents can be found at every step of the biomedical research process, thus enabling future

drug development in a shorter time frame and saving important resources.

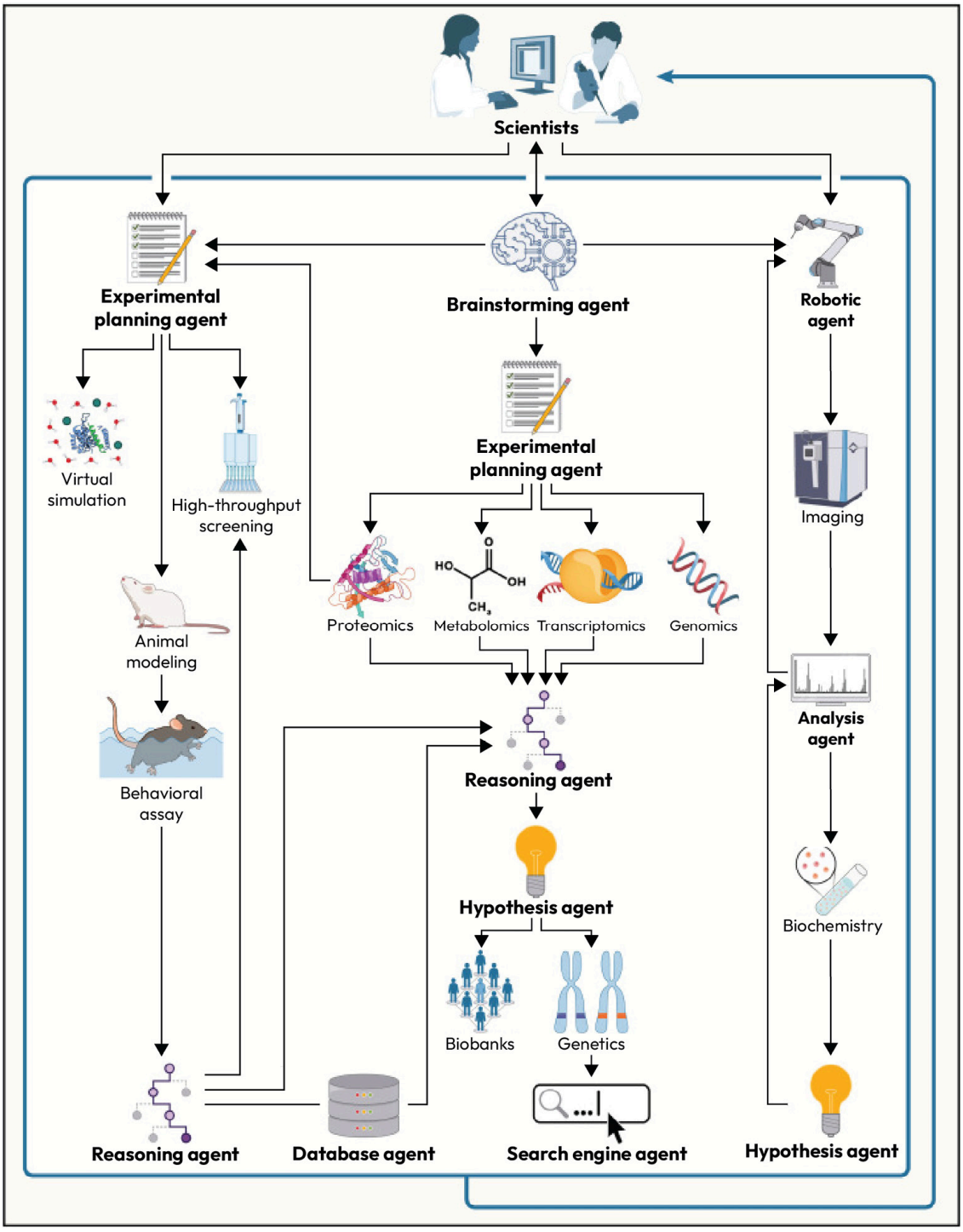


Figure 11.1 – Empowering biomedical research with AI agents (https://www.cell.com/cell/fulltext/S0092-8674(24)01070-5)

Biomedical AI agents

ChemCrow is an example of this type of agent, defined for a specific case and domain. The reasoning of the system is limited to the specific tasks; the agent must use the experimental data and existing knowledge. It is

the researcher who defines both the hypothesis and the tasks; the system only has to complete them. Level 1 can be considered orchestrators under the supervision of a human being.

For example, ChemCrow has demonstrated concrete outcomes in research automation: according to a study published in *Nature Machine Intelligence*, ChemCrow autonomously planned and executed the synthesis of an insect repellent and three organocatalysts, and guided the screening and synthesis of a novel chromophore (*Nature Machine Intelligence*, 2024). Additionally, by integrating 18 specialized tools, ChemCrow has streamlined complex chemical research processes, significantly increasing efficiency and accessibility for both expert and non-expert users (*ScienceDaily*, 2024).

Most agent approaches are based on the use of a central LLM. An LLM is pre-trained with general knowledge and then aligned to human preferences to make the most of its knowledge and the skills it has learned during pre-training. The biomedical field requires specialized expertise and knowledge. Therefore, various experiments have often been conducted where an LLM has been fine-tuned to specialize in medicine (e.g., BioGPT, NYUTron, and MedPalm). This approach is clearly expensive, and a model becomes outdated quickly (thousands of papers are published every day). So, different approaches have been sought in which it is not necessary to conduct repeated rounds of fine-tuning.

One option is to try and use one model (one LLM) but with different professional expertise (assigning a specific role at each round). The idea is to use one model, but craft prompts to assign a role to the LLM (biologist, clinician, chemist, and so on). There are also other alternatives, for example, using instruction tuning to create an expert for a domain (so rather than aligning the model on specific knowledge, align it on specific tasks that would be an expert's). For example, we can ask a model to perform a task (*Write a sequence for a protein X that has a function Y*) or provide it with a specific role (*You are a biologist specializing in proteomics; your task is: write a sequence for a protein X that has a function Y)*. A complex task can be performed by more than one specialist; for example, we can provide the model with the task directly (*Identify a gene involved in the in-*

teraction of the Covid virus with a respiratory cell; design an antibody to block it) or break it down into several subsequent tasks (a first task with a first role such as *You are a professional virologist with expertise on the Covid19 virus; your task is: identify a gene involved in the interaction of the Covid virus with a respiratory cell* and then assign the model a second task: *You are a computational immunologist with expertise in designing blocking antibodies; your task is: design an antibody to block it*). In contrast to the previous approach to learning, a methodology (solving tasks) does not quickly become outdated like domain knowledge. Other authors suggest that one can instead simply use in-context learning.

This strategy means providing the model in context with a whole range of information that would be needed to play the role of a specialist (specific information about the role the model is to impersonate: definition, skills, specific knowledge, and so on). This strategy is very similar to assigning a role by prompt, but we give much more information. Although these prompts are full of information and instructions, the model does not always follow them. Also, it is difficult to describe in a prompt what a specialist's role is. So, an additional strategy is that the model independently generates and refines the role prompt.

Agents may therefore have different tools at their disposal and different purposes. The rationale for this multi-role approach is that an LLM does not have a deep understanding of planning and reasoning but still shows acquired skills. So, instead of one agent having to handle the whole process, we have a pool of agents where each agent has to take care of a limited subtask. Typically, in addition to the definition of different types of agents, there is also the definition of working protocol (for example, in the virtual lab, in addition to agents, a protocol of team and individual meetings was defined).

In any case, although there is so much expectation about a multi-agent approach where there is an LLM acting with several people, some studies give mixed results. In fact, some authors say that what are formally called "personas" (assigning a role to an LLM) do not give a particular advantage, except in rare cases. In any case, to date, it is necessary for these

prompts to be precisely designed to be effective (and it is a laborious, trial-and-error process).

Since LLMs have good critical thinking skills, it has been suggested that they can be used in brainstorming. Although LLMs have no reasoning skills and limited creativity, they can conduct a quick survey of the literature. Agents can then be used to propose ideas, evaluate the best, refine and prioritize, provide critique, and discuss feasibility. One interesting possibility is to use a pool of agents where each agent has different expertise, which mimics the brainstorming discussion process.

Different frameworks can be created where agents interact with humans or with each other. For example, leveraging critique capabilities can facilitate the creation of agents with distinct goals to foster debate. One group of agents could focus on critiquing and challenging ideas, while another could aim to persuade and advocate for their viewpoints. Each agent could have different expertise and have different tools at their disposal. This approach, therefore, evaluates a research proposition from different perspectives. A research idea can then be viewed as an optimization problem where agents try to arrive at the best solution. In addition to a setting where agents are competing, the possibility of cooperation can also be exploited. Agents provide feedback sequentially on a proposition with the purpose of improving an idea. The two frameworks are not necessarily opposites but can be reconciled in systems where each idea goes through feedback loops and critique. Because the frameworks are organized with natural language prompts, multi-agent systems provide unique flexibility.

Similarly, it is not necessary that all agents be equal peers; hierarchical levels can be organized. For example, one agent may have the role of facilitating discussion or having greater decision-making weight. In the virtual lab, there is an agent that has the role of principal investigator, which initiates the discussion and has decision-making power. Thus, multiple decision-making levels can be instituted that are managed by sophisticated architecture.

Note that agents can then design experiments and, conjugated with experiential tools, these experiments can be accomplished. This would pro-

vide a new level of capability, toward a process that becomes end to end.

In this regard, Gao (https://www.cell.com/cell/fulltext/S0092-8674(24)01070-5) defined three levels of autonomy for an agent system in biomedical research:

- **Level 0**: A machine learning model is used as a tool by a researcher. The researcher defines the hypothesis, uses the model for a specific task, and evaluates the output. *Level 0* systems are tools such as models for making predictions in the biological field.
- **Level 1**: This is also defined as *AI agent as a research assistant*; the researcher defines a hypothesis, specifies the tasks that need to be conducted to get to the goal, and the agent uses a restricted set of tools. ChemCrow is an example of this type of agent, defined for a specific case and domain. The reasoning of the system is limited to the specific tasks; the agent must use the experimental data and existing knowledge. It is the researcher who defines both the hypothesis and the tasks; the system only has to complete them. *Level 1* can be considered orchestrators under the supervision of a human being.
- **Level 2**: Also referred to as *AI agent as a collaborator*, the system helps a researcher redefine the hypothesis thanks in part to its large set of tools. Despite its contribution to the hypothesis, its ability to understand scientific phenomena and generate innovative hypotheses remains limited. What differentiates it from *Level 1* is participation in hypothesis improvement and task definition to test it.
- **Level 3**: This is the last level and is defined as *AI agent as a scientist*. In this case, an agent must be able to develop and extrapolate novel hypotheses and define links between findings that cannot be inferred solely from the literature. A *Level 3* agent then collaborates as an equal to a researcher or can propose hypotheses on its own, defines tasks to test hypotheses, and completes them.

To date, we have no agents beyond *Level 1*, and we will probably need new architectures and training systems for *Levels 2* and *3*. *Level 0* is then a set of tools that are used by researchers but lack any autonomy. A *Level 1* agent can write code to conduct a bioinformatics analysis to process data, conduct statistical analysis, or use other tools. A *Level 1* agent uses

Level 0 tools to carry out these tasks, allowing it to test a hypothesis. A *Level 2* agent should not just perform narrow tasks on human indications but should be able given an initial hypothesis to refine it, decide, and perform tasks autonomously. We expect a *Level 2* agent, after being given the hypothesis, to be able to also refine experiments, and then critically evaluate to maximize a goal. A *Level 3* agent, on the other hand, should collaborate with humans to generate hypotheses, and can practically be considered its peer. A *Level 3* agent should be able to evaluate existing challenges and anticipate future research directions. In addition, a *Level 3* agent should integrate with experimental platforms to be able to conduct the entire process end to end.

AI agents in other sectors

In this section, we will discuss how LLM agents are having and will have a global impact across a range of industries.

Physical agents

Physical AI agents (for example, robots) are LLM agents that are capable of navigating the real world and performing actions. Thus, they can be considered systems that are embodied and integrate AI with the physical world. LLMs in these systems provide the backbone for reasoning and contextual understanding. On this backbone, other modules such as memory, additional skills, and tools can be added.

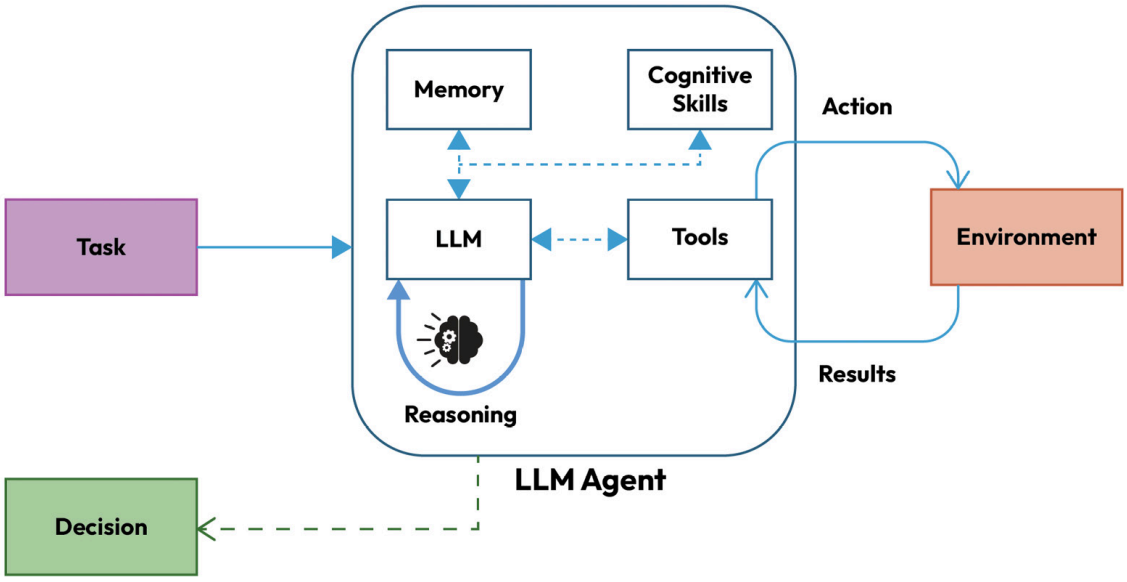


Figure 11.2 – LLM-based agent (https://arxiv.org/pdf/2501.08944v1)

Unlike a virtual agent, a physical AI agent must also understand and adapt to physical dynamics such as gravity, friction, and inertia. Being able to understand physical laws allows it to be able to navigate the environment and perform tasks.

There are several advantages to using an LLM for a physical agent:

- **Human interaction**: LLMs allow humans to interact more easily through the use of natural language. In addition, the use of LLMs allows for better communication and better management of emotions, allowing for easier acceptance. Likewise, people are already accustomed to collaboration with LLMs, thus predisposing users to collaborate more easily with robots to solve problems, generate plans, and perform tasks.
- **Flexibility and adaptation**: LLMs today are multi-purpose with generalist capabilities, which allows them to adapt more easily to different tasks and circumstances. In addition, for specific tasks and environments, LLMs can be fine-tuned to acquire new skills and knowledge needed to operate in different environments. LLMs also have reasoning skills and the ability to find information; this knowledge and these skills acquired during pre-training can be used to solve tasks for which they were not programmed. In addition, LLMs can be guided to perform a task through natural language, making it easy to explain to robots the tasks they need to accomplish.
- **Multimodal capabilities**: Today, several LLMs are capable of taking different types of modalities as input. This capability allows them to integrate information from different types of sensors, so they can understand their surroundings.

In recent years, the idea of combining LLMs and robots has already been explored. For example, PaLM-SayCan was an experiment in which they used Google PaLM to command a robot. Later, Google used a PaLM-E model, which is itself multimodal. In addition, new alternatives are being tested today in which **reinforcement learning** (**RL**) is used to improve the interaction of LLMs with the environment.

Several challenges remain at present for robots controlled by LLMs:

- **Datasets and training**: LLMs require extensive training with large amounts of data. Collecting these datasets is not easy; to date, there are no quality datasets to train a robot in an environment (datasets that require large amounts of images and text). A robot would have to be trained with task descriptions and how to perform them, making it expensive to acquire these multimodal datasets. Using RL requires that you acquire datasets in which you have information about the actions taken by the system and the effect on the environment. Datasets used for one task may not be useful for training in another. For example, a dataset used for training a dog robot cannot be used for training a humanoid robot). Robot training requires interaction with the environment; this is a laborious and time-consuming process. Efforts are being made to overcome this problem with the use of games and simulations. However, this alternative is a simplification of the real environment and may not be enough.
- **Structure of the robot**: A robot can be of an arbitrary shape. Today, motion robots are designed with human shape, but this is not strictly necessary. In fact, robots for particular applications might have different shapes. For example, a robot thought of as a chef might have a better shape if designed for its specific environment.
- **Deployment of the LLM**: The optimum in these systems is to place an LLM inside the robot. Deployment inside the robot is one of the limitations of current LLMs. Many LLMs require considerable hardware resources (different GPUs for a single LLM), which makes deployment inside a local brain not feasible. In contrast, today, the robot's brain resides in the cloud. This obviously has several limitations, especially when there is signal loss.
- **Security**: LLMs have biases and misconceptions that result from pretraining. In addition, LLMs can also hallucinate or commit errors. These factors can manifest themselves in errors while performing a task. An LLM who can control physical actions could then cause harm. For example, a robot could burn down a house while cooking. At the same time, LLMs can be hacked, posing the risk of private data leakage or intentional damage.

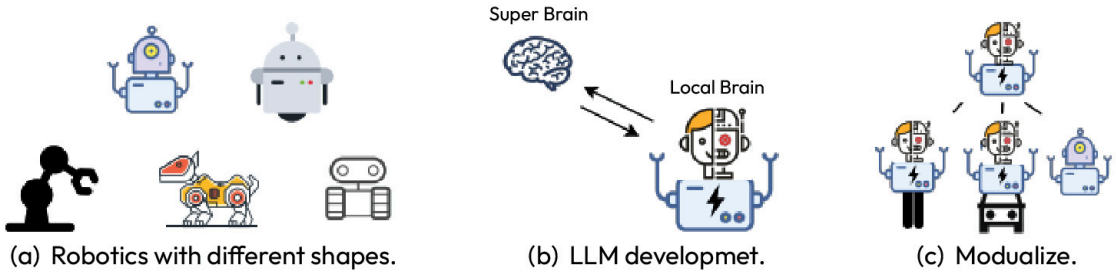


Figure 11.3 – Challenge in embodied intelligence (https://arxiv.org/pdf/2311.07226)

LLM agents for gaming

LLM-based AI agents for gaming are another interesting frontier, where the reasoning capabilities of the model are used to interact with the environment (the game). In general, a framework dedicated to gaming requires a set of components such as an LLM, memory, and tools to interact with the game. Often, the system is trained using RL (where a game is an episode). An LLM can then analyze the moves conducted in previous games and reason about what the best action is.

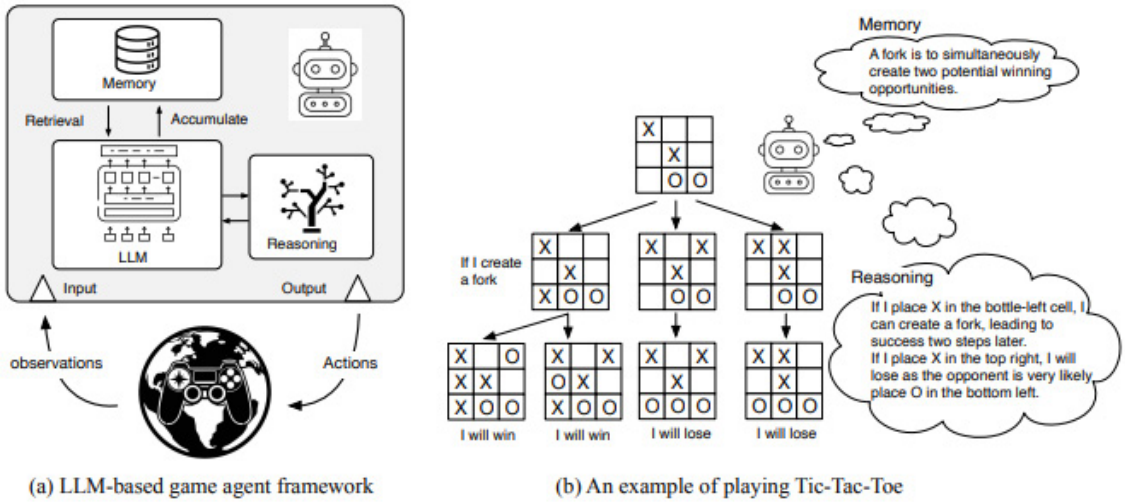
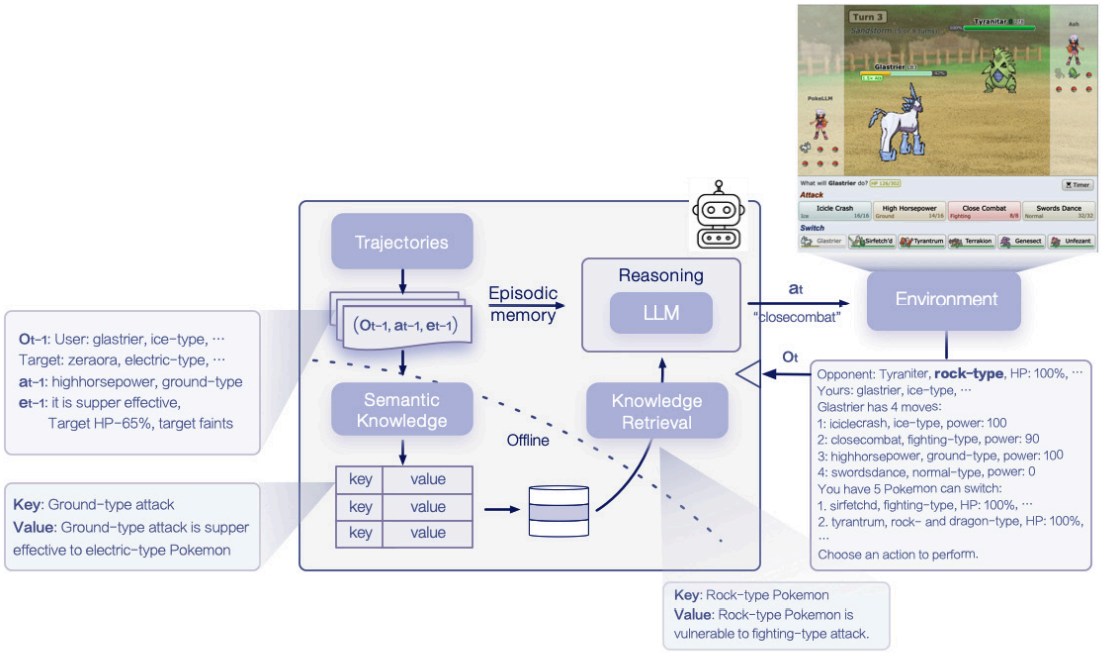


Figure 11.4 – Overall framework for LLM-based game (https://arxiv.org/pdf/2404.02039)

Especially today, many games are quite complex and there is sophisticated interaction with the environment and other characters. An LLM can then reason about the richness of textual information (object descriptions, task descriptions, dialogues with characters, etc.) to decide on a plan of action or strategy. For example, in Pokémon battles, each player has several Pokémon of different species. Each species has different abilities and statistics; knowledge of the game is necessary in order to win a

battle. Using an LLM can allow you to leverage the model's implicit knowledge to be able to select an effective strategy (such as using an electric attack does not bring damage to a ground-type Pokémon). In addition, an LLM can exploit techniques such as chain-of-thought (CoT) to integrate different elements into action choices (especially if it has to think several moves ahead).



(a) An example of playing Pokémon Battles

Figure 11.5 – Use of semantic knowledge for devising an effective strategy (https://arxiv.org/pdf/2404.02039)

LLM-based agents are an interesting prospect for the game because they could enrich the players' experience. For example, LLMs could create characters who discuss more naturally with players, provide hints during the game, cooperate with them, and guide them through the adventure. Or they could be used to generate antagonists that are more complex and match the player's level.

Web agents

Web agents are AI agents designed explicitly to interact with the web and assist humans in tedious and repetitive tasks. Thus, the purpose of these agents is to automate these tasks and improve productivity and efficiency. Again, the brain is an LLM, which allows reasoning and task understanding to be conducted. The architecture of a web agent is similar to that

seen in this book. A web agent has a module dedicated to perception (input from the web), reasoning (LLM), and a module dedicated to interaction with the web. The perception module requires interaction with the web via either HTML (text-based agents that read the HTML document and process it) or via screenshots of websites (use of multimodal LLMs). Once an LLM receives a task, it can then browse the web, schedule subtasks, retrieve information from memory, and execute the plan.

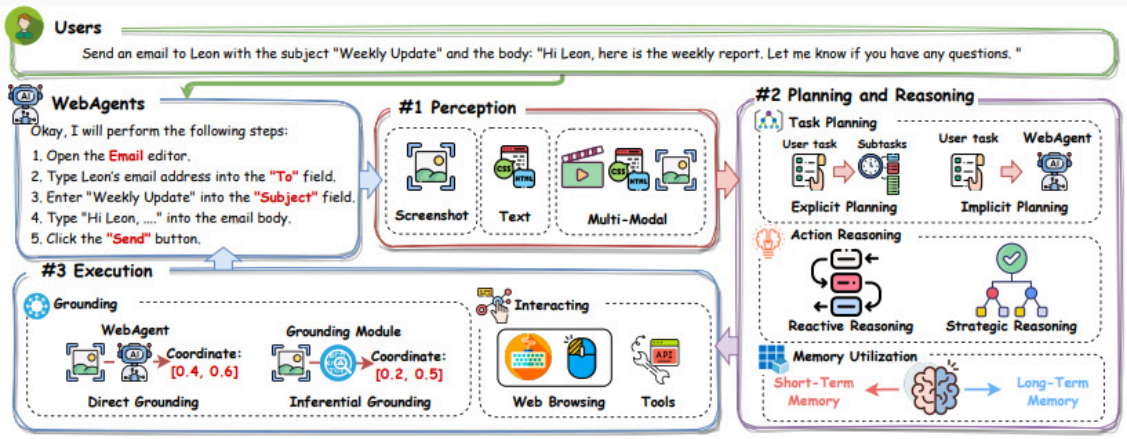


Figure 11.6 – Web agent framework (https://arxiv.org/pdf/2503.23350)

AI agents are a new frontier for AI, one that is poised to have a rapid practical impact. Despite their potential, several challenges and issues remain, which we will address in the next section.

Challenges and open questions

In this section, we will address several open questions about both agents and the capabilities of LLMs. Despite advances in the field, several points remain to be resolved for the safe use of AI agents.

Challenges in human-agent communication

Once they are deployed in the real world, agents can perform actions that lead to problematic failures. For example, a shopping agent might spend money unexpectedly or inadvertently leak sensitive information. Coding agents might execute or produce viruses, delete important files, or push repositories into production that are full of bugs. Communication with the user is key to avoiding such problems. The use of agents should be based on two key principles: transparency and control. Indeed, there

must be an alignment between the user's goals and the agent's behavior; the user must then be able to control the process and have access to its progress. Communication between humans and agents allows us to advance these two principles, but some open challenges remain.

Modern agents are not yet completely perfect and can make mistakes (especially for goals that are complex or include several steps). Therefore, it is important that we can verify the agent's behavior, both the result of its work and that it has understood the task. Therefore, a way must be found to verify that the agent has understood the goal and that its plan and actions are directed toward this goal. Verifying that the agent has truly understood the goal allows us to avoid costly errors and save computation and time.

In addition, LLMs have a component that is stochastic. This component arises from the probabilistic nature of the model output functions (stochastic decoding) and the complex natures of interactions that can evolve during the task (unanticipated events). Therefore, the output and behavior of the model may not be consistent. Even in a deterministic setting (temperature 0), changes in the environment during task execution may lead to unexpected or unintended results. Inconsistencies may also emerge from the outdated model knowledge or imperfect world model present within an LLM. For example, an agent might buy an item that is out of budget or different from a user's needs, due to misalignment of its knowledge of the real world.

Similarly, interactions with the user and the outside world generate a great deal of information. This broad context is important for directing the agent's behavior, which can then be learned from past interactions. Although this context is fundamental to being able to perform the task effectively, it risks becoming far too wide and manageable over time. At the same time, modern LLMs have a noise problem and struggle to find relevant information when it is scattered in unnecessary detail. Therefore, effective ways must be found for an agent to focus on the relevant part of the last interaction with the user. Also, some of the information should not be able to be reused (privacy and ethical concerns), so one would

need to find an easy way to manage, edit, and remove the past information.

What we have discussed are the general challenges of user-agent communication. We can also define open challenges that are in the communication between user and agent, and vice versa. First, we need to make sure that we can design agents that enable effective communication by the user by addressing these points:

- **Clear goal acquisition**: The focus of the system is for the agent to understand the goal and for the user to be able to provide it clearly. To avoid costly mistakes, we need to design agents for which users can define goals unambiguously. Some possibilities have been studied in some areas: sets of logical rules and the use of formal languages. To make this technology usable for everyone, we need to use natural language. Natural language is rich in both nuance and ambiguity, however, and allows complex goals to be defined with vague and incomplete definitions. Hence, mechanisms must be defined to disambiguate unclear goals or allow the agent to infer from context (or past interactions).
- **Respect for user preferences**: One can achieve a goal with several paths, but some are more optimal than others (both for efficiency and for respecting a user's preferences). User preferences may not be aligned with LLM values (during post-training, the model is aligned with human preferences, which do not necessarily reflect the preferences of a general user but only of a selected pool of annotators). For example, if a user requests a route, they may prefer a more eco-friendly means of transportation. The agent should adhere to these preferences when possible, or interrupt the process to inform the user when it cannot. Model alignment may be one possible approach to take user preferences into account. However, current alignment approaches primarily consider aggregate preferences, and methods for accommodating individual preferences remain undeveloped. In more general terms, an agent can also achieve a goal by generating harm (even in an unintended way), and this risk is greater if it has the ability to use tools.

- **Incorporating feedback**: We know agents are error-prone, and while we can develop strategies to reduce errors, completely eliminating them may not be possible. An agent might continue to use suboptimal tools (not understanding the goal or setting the wrong plan) in repeated interactions, frustrating the user. One way to correct this behavior is to provide feedback from the user. There is now research on how to incorporate this feedback and how to represent it in a more effective form for the agent (e.g., turn it into first-order logic).

There are also challenges that are associated with how the agent communicates to the user, especially pertaining to their capabilities, what actions they take or will take, goal achievement, and unexpected events:

- **Capabilities of the agent**: The user must be able to understand the full capabilities (and limitations) of an agent in order to conduct informed decision-making. It should be clear what information the agent has access to, how it will use this information, how it can modify the external environment, what tools it has access to, and whether it can connect to the internet.
- **What actions the agent will take**: To solve the goal, an agent can detail a complex plan, which can be particularly costly (time, resources, or money) and may violate some of the user's preferences. The user then should be aware of the actions an agent takes and be able to provide feedback. Of course, an effective form of communication must be found to avoid irrelevant details being communicated and the user not fully understanding the agent's actions. In addition, it should be clarified whether some actions require the user's explicit approval.
- **Monitor progress**: For an agent moving in a dynamic environment, a plan to complete a task requires several steps; it is useful for a user to be aware of what the agent is doing and whether it is necessary to modify the process or stop it. An agent conducting multiple actions at the same time could lead to unexpected and harmful behavior. For example, an agent who builds news reports and invests in the market might read fake news and conduct a series of bad investments.
- **Changes in the environment and side effects**: An agent must monitor the changes in the environment or potential side effects of its operations. Take, for example, an agent tasked with buying a product on-

line at the lowest price available. The agent could search online and find the product at a very competitive price and order it. However, the offer might require a subscription or other hidden costs that would make the purchase much more expensive than the user's preferences or budget. The user must be aware of the side effects that are generated by the agent's behavior.

- **Goal attainment**: The user specifies a goal, and the agent plans actions and executes them. At the end of this process, the user must be clear whether the agent has achieved the goal or not (or partially). Thus, a way is needed to evaluate that a goal has been achieved. For example, the goal might be to buy the cheapest possible cell phone with a certain type of performance. The agent could lead the purchase, but we need to assess whether the agent has met the other conditions as well. Thus, we need a way to verify that the goal has been fully and satisfactorily achieved.

Communication with agents is a complex but critical topic. Miscommunication can lead to system failure and is an important point to consider. In this section, we have provided a list of important elements to evaluate user-agent communication from different perspectives. In the next subsection, we will see whether or not the use of multi-agents is superior to the single agent. Some studies question this perspective.

No clear superiority of multi-agents

As mentioned earlier, an LLM-based agent can be identified as an entity that has an initial state (usually a description in the prompt specifying its initial state), can track what it produces (state), and can interact with the environment through the use of tools (action). A **multi-agent system (MAS)** is defined as a collection of agents that interact with each other in a coordinated manner to solve a task.

MASs are an extension of single-agent systems, designed to create a more sophisticated framework capable of addressing complex problems. Obviously, this means a higher computational cost (more LLM calls in inference). This higher computational cost should be justified by a substantial performance gain. In fact, some studies show that this is not the case.

MASs offer only marginal gains in performance compared to single-agent systems.

It is useful to outline the architectural trade-offs between single-agent and multi-agent designs. While MASs offer potential advantages in modularity and parallelism, they also introduce additional complexity, coordination overhead, and cost. The following table summarizes the key differences:

	Single-Agent Design	**Multi-Agent Design**
Cost	Lower: fewer inference steps and less orchestration	Higher: more agents, and more LLM calls and tool usage
Latency	Generally lower, streamlined single flow	Potentially higher due to inter-agent communication
Fault tolerance	Lower: failure in the agent often breaks the system	Higher: failures can be contained within individual agents
Modularity	Monolithic and harder to extend	Modular: agents can be added or replaced independently
Scalability	Limited: the agent handles all logic	Higher: parallel agents allow distributed problem-solving
Communication overhead	None (internal reasoning)	Significant: explicit agent-to-agent messaging required

Interpretability	Easier: single decision chain	Harder: distributed reasoning may reduce transparency

Table 11.1 – Potential causes of multi-agent system failure

As shown in the preceding table, multi-agent architectures introduce a set of trade-offs that must be carefully balanced. While they offer modularity and potential fault isolation, they often suffer from increased latency, communication overhead, and coordination challenges. These trade-offs are reflected in empirical evaluations of MASs.

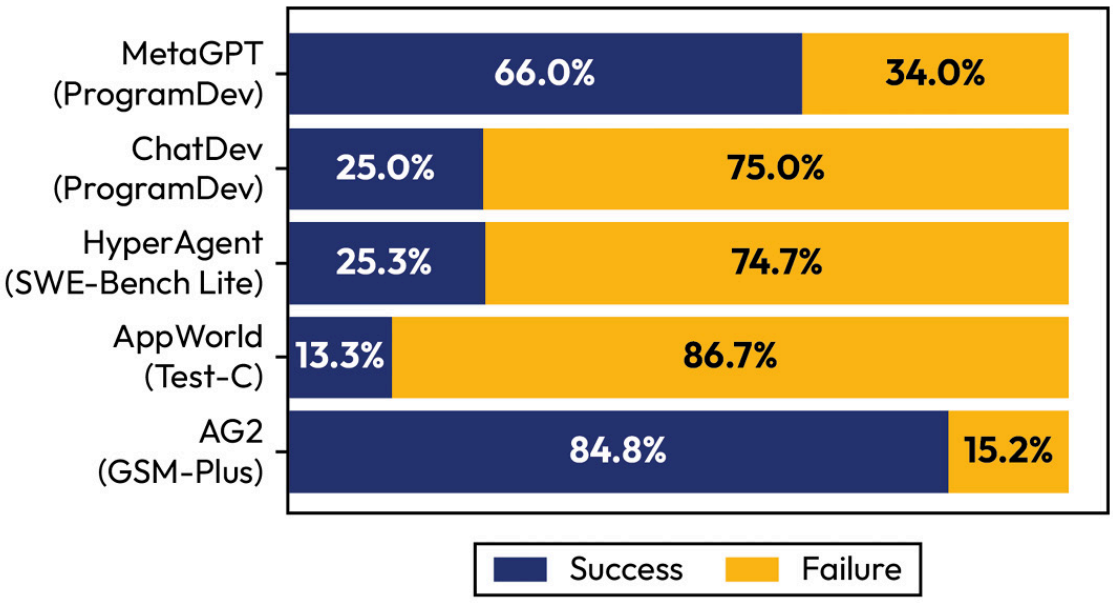


Figure 11.7 – Failure rates of five popular multi-agent LLM systems (https://arxiv.org/pdf/2503.13657)

MASs should bring numerous benefits, such as greater accuracy and the ability to handle more complex tasks, create more complex plans, or find better solutions. If MASs do not bring all these benefits and indeed often fail, we need to understand why. In a recent study, Cemri et al. (2025) set out to conduct a detailed taxonomy of MAS failures with expert annotators by analyzing 150 conversation traces (each averaging over 15,000 lines of text) to identify failures and the causes of these failures. In their work, they identified 14 causes, grouped into 3 main groups:

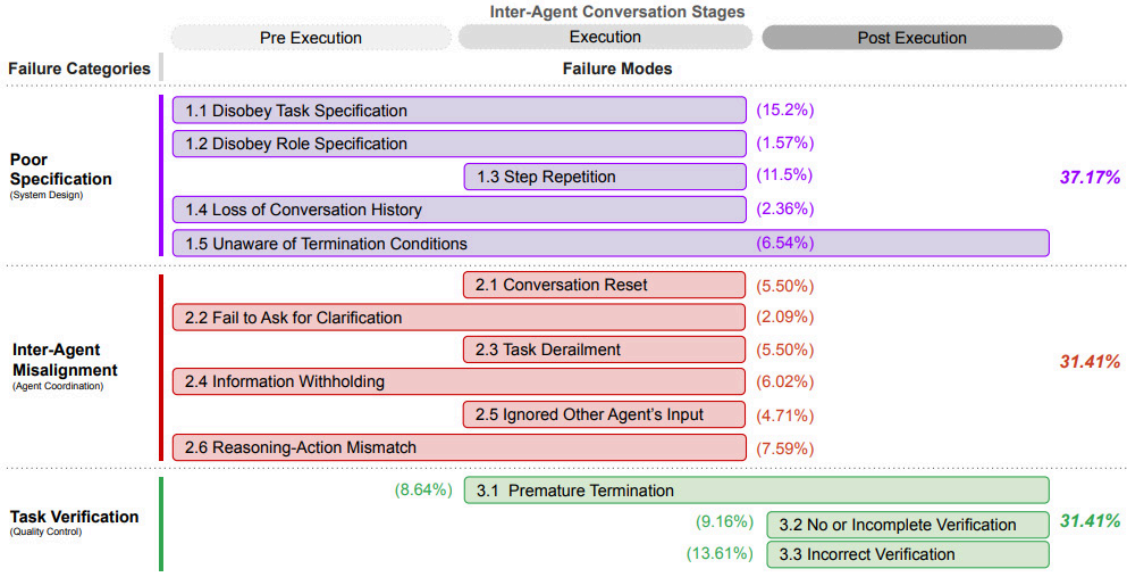


Figure 11.8 – Taxonomy of MAS failure modes (https://arxiv.org/pdf/2503.13657)

The three main categories are thus as follows:

- **Specification and system design failures**: Failure results from deficits in MAS design. For the authors, much of the failure stems from poor choice of architecture, management of conversation between agents, poor task specification, violation of constraints, and poor specification of agent roles and responsibilities. In other words, if the instructions for agents are not clear, the system may fail. Even when instructions are clear, however, the MAS may not be aligned with user instructions.
- **Inter-agent misalignment**: Failure emerges from ineffective communication, little collaboration, conflicting behaviors among agents, and gradual derailment from the initial task. As we mentioned previously, achieving efficient communication between agents is not easy. Therefore, some agents may not communicate efficiently and simply waste resources.
- **Task verification and termination**: A third important category includes failure to complete the task or its premature termination. MASs often lack a verification mechanism that checks and ensures the accuracy, completeness, and reliability of interactions, decisions, and outcomes. Simply put, many systems do not include a dedicated agent (or other mechanism) to monitor the process and verify that the task was successfully executed.

The results of their investigation showed that none of the causes are prevalent but are equally distributed across systems. In addition, some causes are correlated, producing a kind of ripple effect. For example, wrong architecture design can cause inefficient communication between agents.

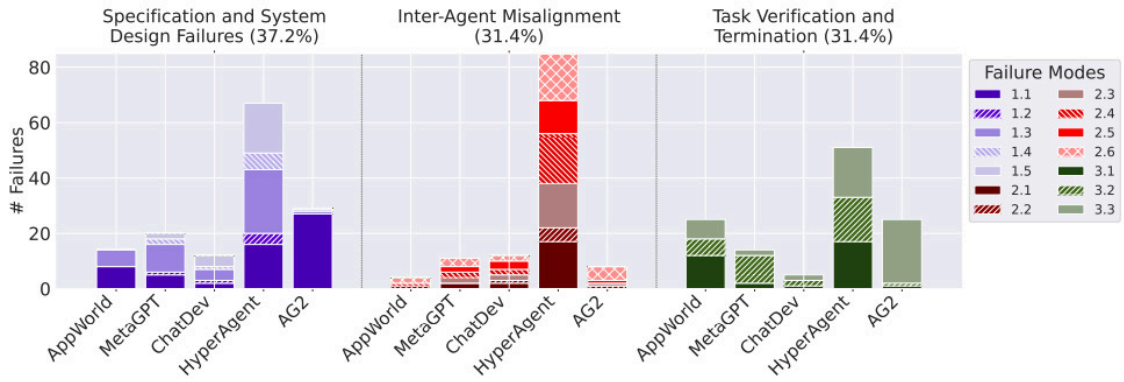


Figure 11.9 – Distribution of failure modes by categories and systems (https://arxiv.org/pdf/2503.13657)

The results of this work clearly show that failures can be avoided through more careful design. Improving prompts, agent communication, and adding an agent (or other verifier mechanism) allow for noticeably improved performance and lower risk of failure. In two case studies, the authors show how this is the case. On the other hand, these suggestions are not enough to solve all agent problems but will be further technical progress.

In addition to the system itself, many of the limitations of agents also stem from the agent itself (i.e., the model that is used for agents). In the next subsection, we will discuss the reasoning limitations of LLMs.

Limits of reasoning

Reasoning is a fundamental cognitive function of human beings, and it is difficult to give a precise definition. Wikipedia defines reasoning this way: "*Reason is the capacity of consciously applying logic by drawing valid conclusions from new or existing information, with the aim of seeking the truth. It is associated with such characteristically human activities as philosophy, religion, science, language, mathematics, and art, and is normally considered to be a distinguishing ability possessed by humans*."

For a long time, it was said that only human beings are equipped with reasoning. Today, however, it has been shown that primates, octopuses, and birds also exhibit basic forms of reasoning such as making decisions or solving problems. One of the problems with reasoning is the difficulty of being able to evaluate it. Typically, to do this, one assesses the ability to solve complex problems or make decisions. Complex problem-solving requires identifying the problem, dividing it into subproblems, finding patterns, and then choosing the best solution. Decision-making similarly requires identifying problems and patterns and evaluating alternatives before choosing the best solution.

In the case of LLMs, an attempt was made to measure reasoning capabilities through benchmark datasets that assess problem-solving ability (such as GLUE, SuperGLUE, and Hellaswag). Today, on many of these datasets, humans have been outperformed by next-generation LLMs. These new reasoning capabilities would be mainly due to three factors:

- LLMs performing well in all the benchmarks dedicated to reasoning. These benchmarks contain math or coding problems that require reasoning skills. The results in these benchmarks suggest that LLMs are capable of reasoning.
- The emergence of new properties with increasing parameters, number of tokens, and compute budget.
- The use of techniques such as CoT, which allows the model to fulfill its potential.

There are those who question this view, claiming that there are alternative explanations for the performance achieved in these benchmarks. After all, many authors regard LLMs as nothing more than mere stochastic parrots. Jiang, in 2022 (**https://arxiv.org/pdf/2406.11050**), suggested that the models are merely pattern-matching machines: "*A strong token bias suggests that the model is relying on superficial patterns in the input rather than truly understanding the underlying reasoning task*."

In the same study, it was observed that LLMs fail to generalize when they encounter new examples that exhibit patterns different from those seen in the pre-training phase. If we change tokens in the examples, pattern mapping fails (a transformer, through in-context learning, tries to find ex-

amples in its knowledge that are similar to the problem posed by the user). When the model fails to find examples, the model fails to solve the question. This fragility and dependence on training examples would explain why the model succeeds in solving complex problems (it finds patterns) and fails even with some very simple questions (it does not find examples). This is confirmed by a correlation between the example's frequency in training data and test performance.

For example, when the model is asked to solve the classic "25 horses" graph theory problem, the model succeeds. If the "horse" token is changed to "bunny," the model fails to solve it. The token change is irrelevant to the problem's underlying logic, yet the model fails to solve it because it has difficulty mapping the problem. Both GPT-4 and Claude have significant performance drops due to perturbations in animal names and numbers.

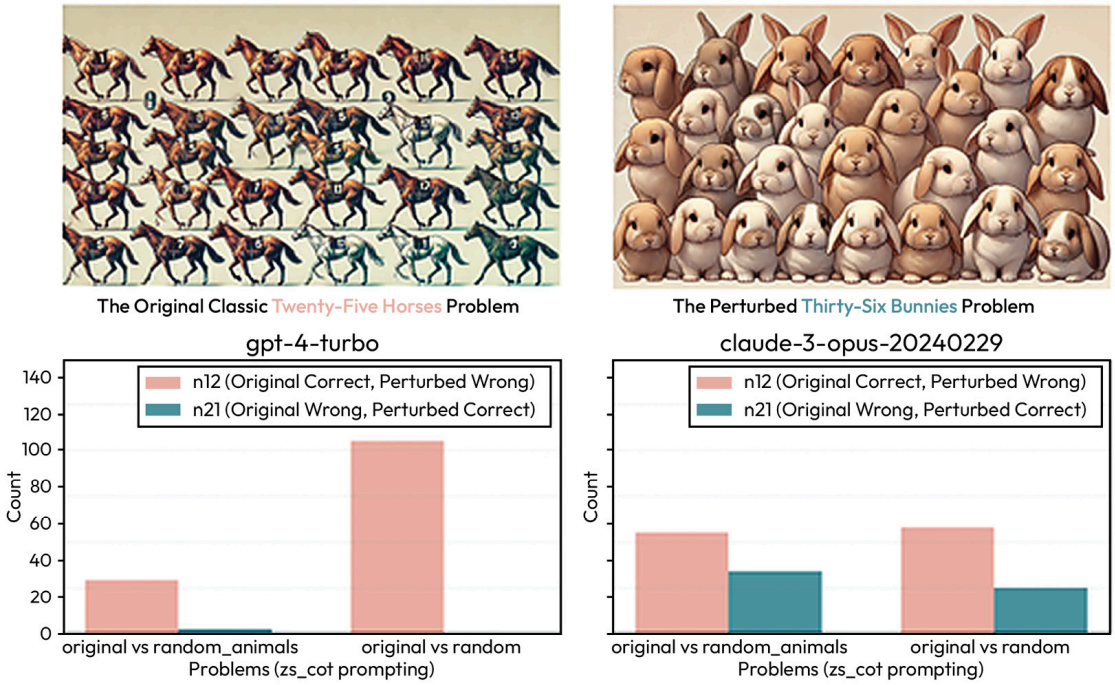


Figure 11.10 – Token bias using the classic problems (https://arxiv.org/pdf/2406.11050)

This phenomenon is called **prompt sensitivity** (a different response to a prompt that is semantically equivalent to another). This is confirmed by the fact that LLMs are sensitive to noise. They are easily distracted by irrelevant context, which makes it more difficult to find patterns. This sensitivity is not resolved by prompting techniques specialized to improve reasoning, suggesting that disturbing pattern-matching activity disrupts

reasoning ability. An example of irrelevant context disrupting the pattern but not impacting actual problem-solving follows:

Original Problem
Jessica is six years older than Claire. In two years, Claire will be 20 years old. How old is Jessica now?
Modified Problem
Jessica is six years older than Claire. In two years, Claire will be 20 years old. *Twenty years ago, the age of Claire's father is 3 times of Jessica's age.* How old is Jessica now?
Standard Answer 24

Figure 11.11 – Irrelevant context disturbs LLMs (https://arxiv.org/pdf/2302.00093)

Some authors suggest that intelligence can be seen as an emergent property. Biological systems naturally tend to become more complex, and this process is driven by natural selection. Evolution has shown an increase in intelligence over time as it promotes the adaptability of various species. Of course, intelligence is not an economic process, and a larger brain consumes a greater amount of resources (metabolic consumption). Loss function could be seen as evolutionary pressure. From this, it would follow that the increase in model capacity (in terms of the number of parameters) would parallel the increase in neurons in animal brains over time, and that loss function would instead be the evolutionary pressure to push these parameters to be used efficiently. By scaling up models and training (parameters and training tokens), intelligence could also emerge in LLMs. Reasoning then is seen as an emergent property that emerges from scaling the models. However, later studies suggest that emergent properties in LLMs can be a measurement error, and with it, the whole theory is related to the emergence of reasoning.

In the next figure, you can see how some properties seem to emerge as the model size increases.

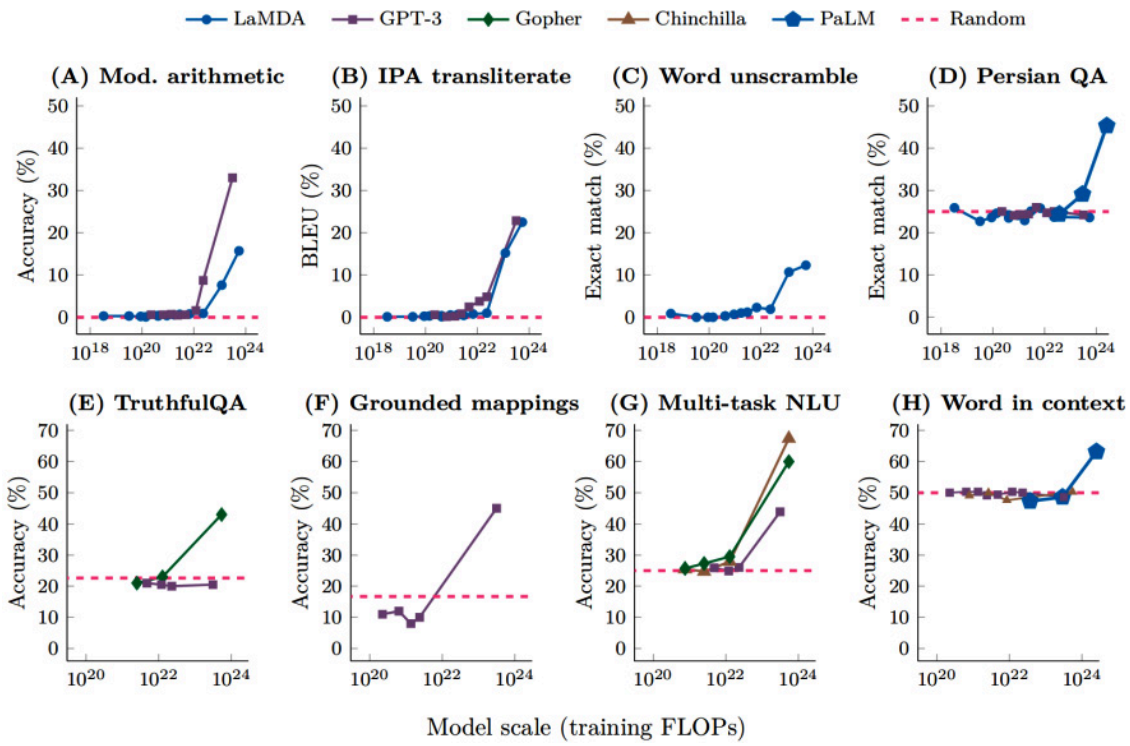


Figure 11.12 – Examples of emerging reasoning properties (https://arxiv.org/abs/2304.15004)

According to other authors, LLMs are capable of reasoning, but it needs to be unlocked. CoT prompting thus helps the model unlock its potential through intermediate reasoning and thus guides it to the correct answer in arithmetic problems. CoT is today's prompt engineering technique and is also used to train deep reasoning models (such as ChatGPT-o1 or DeepSeek R1). In fact, these models are trained on long CoTs that are used to conduct supervised fine-tuning. These models explore different reasoning paths to arrive at an answer, showing high improvements in reasoning benchmarks. However, some studies show that these models suffer from both overthinking and underthinking.

Overthinking is a curious phenomenon in which these models reason longer than necessary when it comes to solving problems that are particularly simple. The model explores different reasoning paths for trivial questions. This indicates that the model is unable to understand which question needs more effort. Underthinking is the opposite, wherein the model may abandon the promising thinking path. This indicates a clear lack of depth of reasoning, where the model does not go all the way to a correct solution.

At the same time, even the benefits of CoT have been questioned (https://arxiv.org/pdf/2409.12183): *"As much as 95% of the total performance gain from CoT on MMLU is attributed to questions containing "=" in the question or generated output. For non-math questions, we find no features to indicate when CoT will help."*

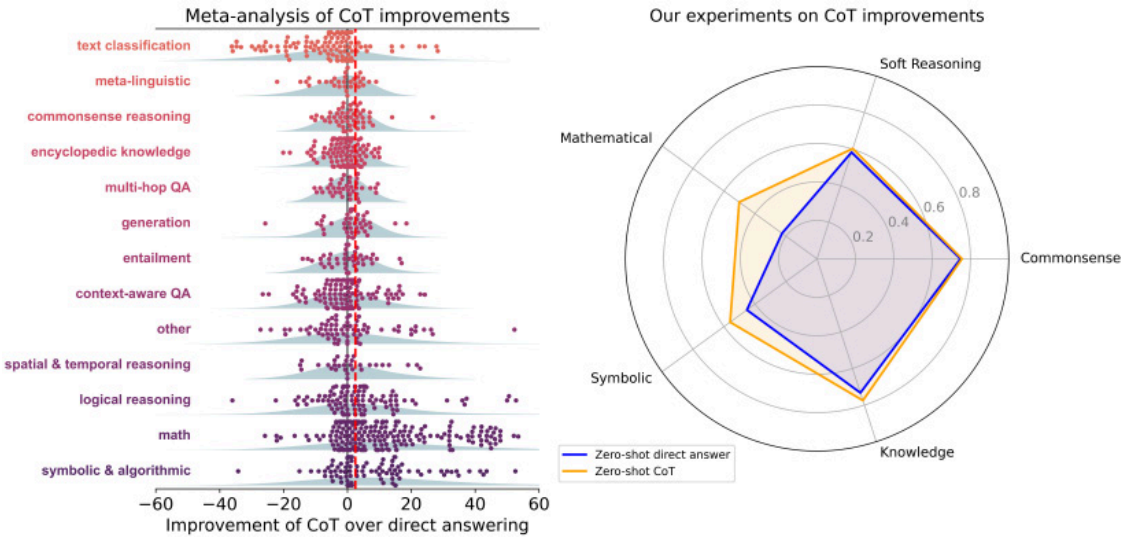


Figure 11.13 – CoT improvements are limited to symbolic and mathematical reasoning (https://arxiv.org/pdf/2409.12183)

CoT would seem to help the model solve problems as it allows it to leverage the skills it learned during pre-training. CoT would simply help develop a plan, but then the LLMs may not be able to execute it. So, CoT can be used to get a plan, but to get the most benefit, an external tool would have to be added (such as a Python interpreter).

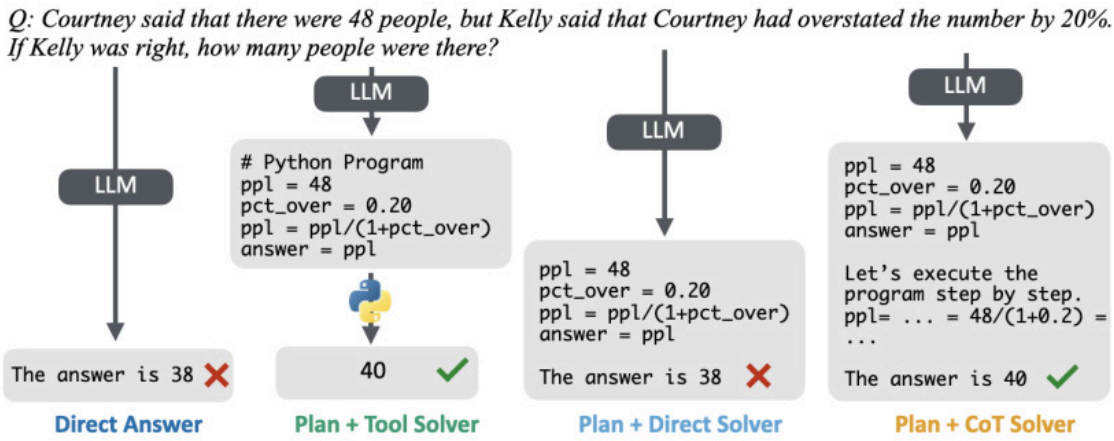


Figure 11.14 – An LLM can devise a plan but needs an external tool to better solve some problems (https://arxiv.org/pdf/2409.12183)

These models are all tested on the same benchmarks as the **Grade School Math 8K (GSM8K)** dataset, which provides complex arithmetic problems

but is at risk of data leakage (considering how many billions of tokens are used to train an LLM, the model may have already seen the answer in the training).

Therefore, in their study, Mirzadeh et al. modified GSM8K, keeping the same questions but making statistical pattern matching difficult. If the model was capable of true reasoning, it should solve it easily; if, instead, it relied on pattern matching, it would fail.

In the following figure, notice how the GSM8K examples are modified to better control the response of the LLM. Using this dataset, we can formally investigate the LLM's reasoning and highlight that state-of-the-art LLMs exhibit significant performance variations; this shows that LLM reasoning is fragile.

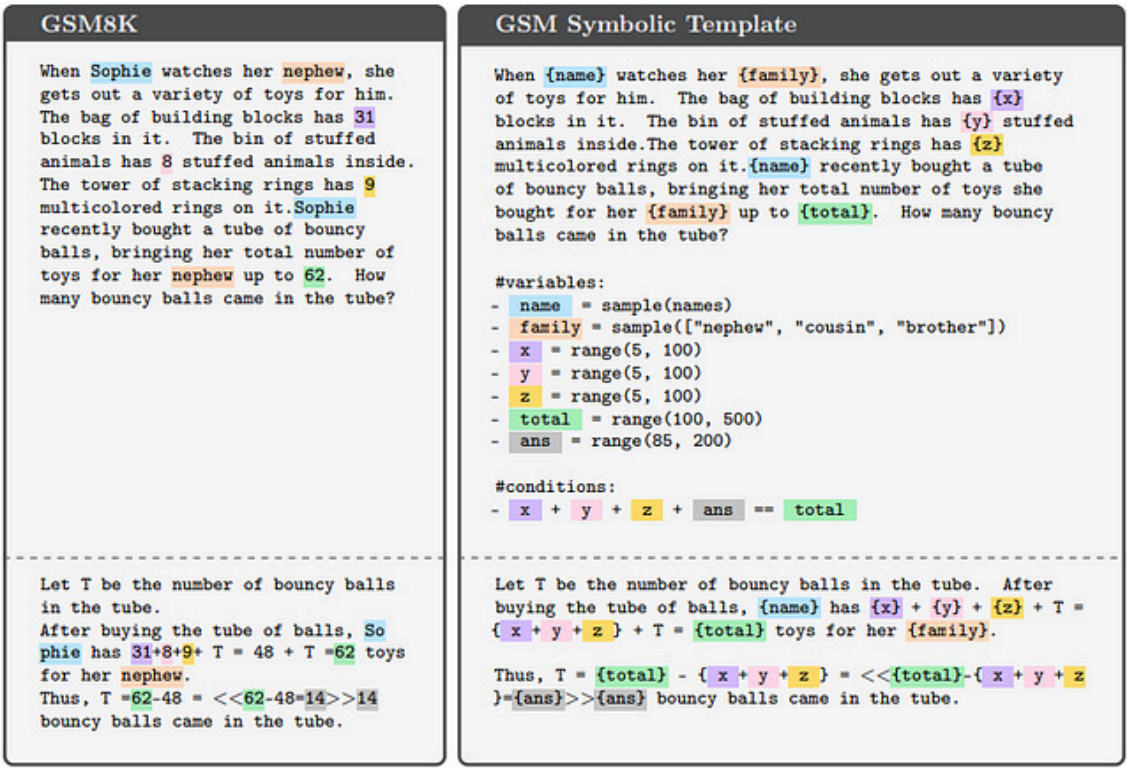


Figure 11.15 – This dataset serves as a tool to investigate the presumed reasoning capabilities of LLMs (https://arxiv.org/pdf/2410.05229)

Testing state-of-the-art LLMs, Mirzadeh et al. found no evidence of formal reasoning in language models. The models are not robust and have a drop in performance when numerical values are changed, and their capabilities degrade sharply as the complexity of the problem increases. The model is, in fact, fooled by added phrases that have no relevance. Instead, the model takes them into account, tries to map them, and some-

times turns them into operations. Mirzadeh et al. suggest that this occurs because their training datasets included similar examples that required conversion to mathematical operations: "*For instance, a common case we observe is that models interpret statements about "discount" as "multiplication", regardless of the context. This raises the question of whether these models have truly understood the mathematical concepts well enough.*"

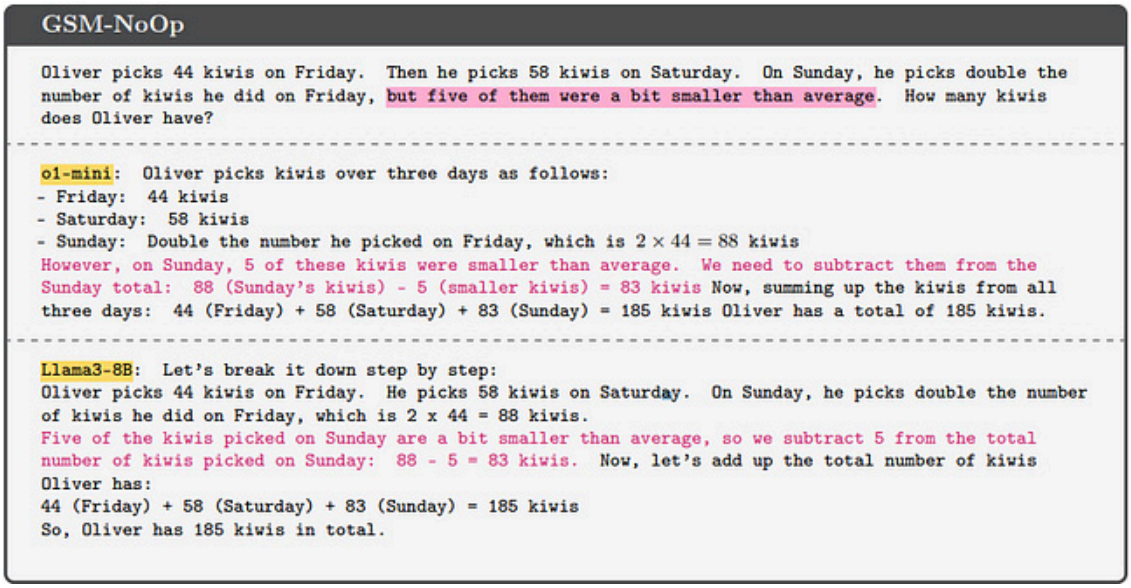


Figure 11.16 – Example of error (**https://arxiv.org/pdf/2410.05229**)

More recent LLMs that have been trained on CoT (such as GPT4-o1) also fail in this task. This suggests that LLMs are elaborate statistical pattern machines but do not possess true reasoning.

Creativity in LLM

Creativity is considered along with reasoning to be one of the skills that makes human beings. If quantifying reasoning is hard, being able to quantify creativity is a much harder task. However, creativity plays a very important role in what makes us human, and it concerns activities such as writing poems or books, creating works of art, or even generating theories and achieving groundbreaking discoveries. That is why the question as to whether an LLM can be creative has been raised.

The problem in investigating the creativity of LLMs is that we do not have an unambiguous definition of creativity. In the field of research, creativity is often used as the definition chosen by Margaret Boden: "*the ability to*

come up with ideas or artifacts that are new, surprising and valuable."
Although this definition is accepted, it is difficult to evaluate its elements:

- **Value**: This is the easiest element to define. For example, code produced by an LLM can be considered valuable if it works in its own way.
- **Novelty**: For an object to be considered novel, it should be dissimilar to what has already been created. For a text, being novel could be considered the difference in output compared to other texts. One definition might be to generate a text whose embedding is distant from other different texts.
- **Surprising**: This is considered one of the most important and difficult elements to define. Random recombination of words can be considered new (or different) but certainly not surprising (nor valuable). *Surprising* is often understood as something new but not a simple variation or recombination.

Boden at the same time described what she thought were three types of creativity with respect to the concept of surprise:

- **Combinatorial creativity**: The combination of familiar elements in an unfamiliar way (such as two genres that have not been combined previously)
- **Exploratory creativity**: The exploration of new solutions in the way of thinking (such as a new narrative style, or a twist to a narrative style that had not been explored)
- **Transformational creativity**: Changing the current narrative style or the current way of thinking

In line with these definitions, several authors have sought to understand whether LLMs can be creative, and if so, what kind of creativity they can manifest. The main problem with this investigation is trying to quantify the creativity of an LLM. One approach is to assess whether the output of LLMs can be mapped to existing text snippets on the web. Human creativity is influenced by previous writers, but when a writer produces original writing, this cannot be mapped to previous writings. If every text generated by an LLM can be mapped to other texts, it is overwhelming evidence of a lack of creativity. In a recently published study, Lu (2024) analyzed

how much of what is produced by an LLM is mappable to texts on the internet. The purpose of this study was precisely to create a creative index and compare LLMs and human beings.



Figure 11.17 – Mapping of LLM output to internet text (**https://arxiv.org/pdf/2410.04265**)

The results of this approach show that humans exhibit greater creativity (based on unique word and sentence combinations) than LLMs. That small amount of residual creativity in LLMs may simply result from stochastic processes and the fact that we do not know the entire pre-training dataset.

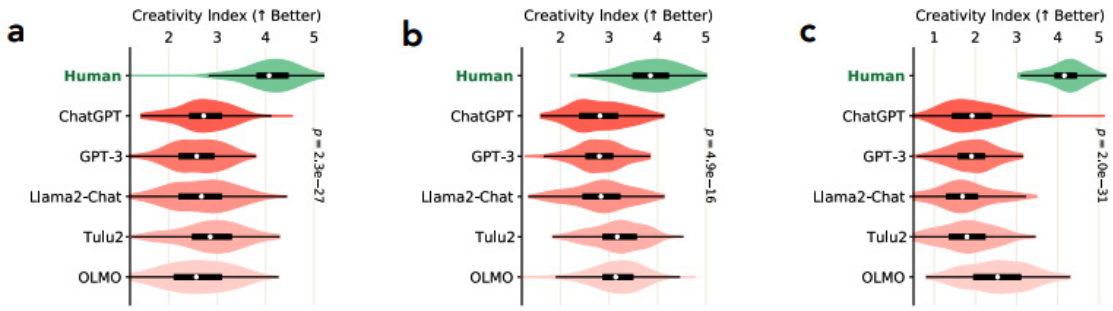


Figure 11.18 – Comparison between the creativity index of humans and that of LLMs (**https://arxiv.org/pdf/2410.04265**)

Lou et al. suggest an interesting analogy: *"Just as a DJ remixes existing tracks while a composer creates original music, we speculate that LLMs behave more like DJs, blending existing texts to produce impressive new outputs, while skilled human authors, similar to music composers, craft original works."*

Despite LLMs being incapable of true creativity, several studies have tried to increase the pseudo-creativity of models (in the long run, LLMs can be particularly repetitive). There are three potential strategies for doing this:

- **Acting on the hyperparameters of an LLM**: The first strategy coincides with raising the temperature of an LLM. Temperature controls the uncertainty or randomness in the generation process. Adjusting temperature impacts model generation, where at low temperatures (e.g., 0.1–0.5), the model generates deterministic, focused, and predictable outputs. Increasing the temperature generates output that becomes less predictable. Beyond 2.0, the process becomes chaotic and the model generates nonsense. So, for applications that require creativity, you can explore higher temperatures but remember that this also generally leads to a reduction in consistency.
- **Conducting additional training for an LLM**: The use of post-training techniques is an avenue that is being explored widely today. Post-training techniques are used for model alignment and to make the model more receptive to performing tasks. Some authors have proposed using techniques that also incentivize the variety of outputs.
- **Prompting strategy**: Use prompts that try to force the model to be more creative. However, prompting strategies do not seem to have great results.

Mechanistic interpretability

Recent advances in AI have meant rapid advancement in model capabilities. Paradoxically, the paradigm of self-supervised learning means that even if models are designed by humans, the capabilities of LLMs are not designed a priori. In theory, a developer only needs to know the process without understanding how the model works, since the desired properties appear during training. In other words, an LLM is not designed for

the properties it shows; these properties were obtained through scaling, and much of how it gets there is unclear. Reconstructing how they appear and the mechanisms behind these abilities is not an easy task, especially after a model of billions of parameters has been trained. These models are considered to be black boxes, and recently, there has been some discussion of how they can be analyzed.

There are several types of interpretability for a model (as described in the following list and figure). Each of these types of interpretability focuses on different aspects.

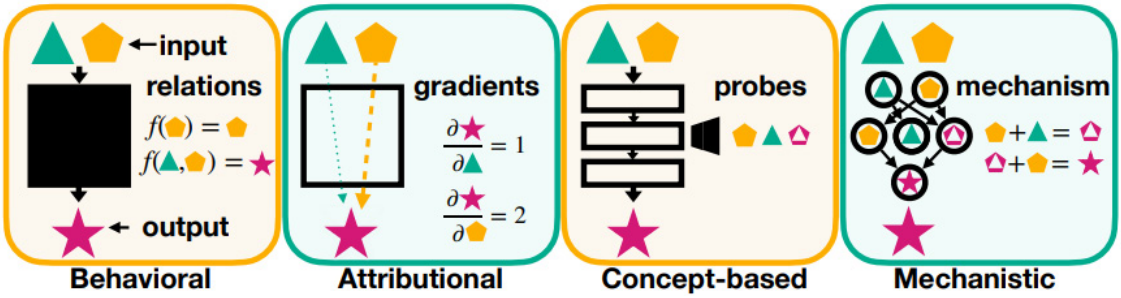


Figure 11.19 – Progressive level of interpretation of a model (https://arxiv.org/pdf/2404.14082)

We can divide the various types of approaches to interpretability into the following:

- **Behavioral**: One considers the model as a black box and is interested in the relationship between input and output. This paradigm considers those classical approaches to interpretability that are model-agnostic.
- **Attributional**: The approaches try to understand the decision-making processes of the model by tracking the contribution of each component of the input and are based on the gradient shift.
- **Concept-based**: Probes are used to try to better understand the learned representation of the model.
- **Mechanistic**: This is a granular analysis of the components and how they are organized, trying to identify causal relationships.

Mechanistic interpretability aims to uncover the internal decision-making processes of a neural network by identifying the mechanisms that produce its outputs. We focus on this approach because it emphasizes understanding the individual components of a model and how they con-

tribute to its overall behavior. This perspective is valuable as it enables us to analyze the model through a comprehensive and transparent lens.

Mechanistic interpretability goes beyond previous approaches because it seeks to identify causal mechanisms to the generalization of neural networks, and thus the decision-making processes behind them. In response to the growth of models and their increased capabilities, there has been a question of how these models acquire these general capabilities, and thus a need for global explanations.

Although LLMs generate text that resembles that produced by humans, this does not mean that the representation of concepts and cognitive processes are the same. This is demonstrated by the fact that LLMs display superhuman performance on some tasks, whereas in other tasks that are simple for humans, they fail miserably. We need a way to solve this paradox, which is through mechanistic interpretability. To try to resolve this dissonance, reverse engineering of LLMs has been proposed. Reverse engineering (a mechanistic interpretability approach) involves three steps: decomposing the model into simpler parts, describing how these parts work and how they interact, and testing whether the assumptions are correct. While mechanistic interpretability aims to uncover the internal logic and causal mechanisms within the network, concept-based interpretability focuses on understanding how models represent high-level, human-understandable concepts and how these concepts contribute to the model's decisions, providing insights into the reasoning behind predictions and bridging the gap between human cognition and machine learning processes. These two approaches are shown in the following figure:

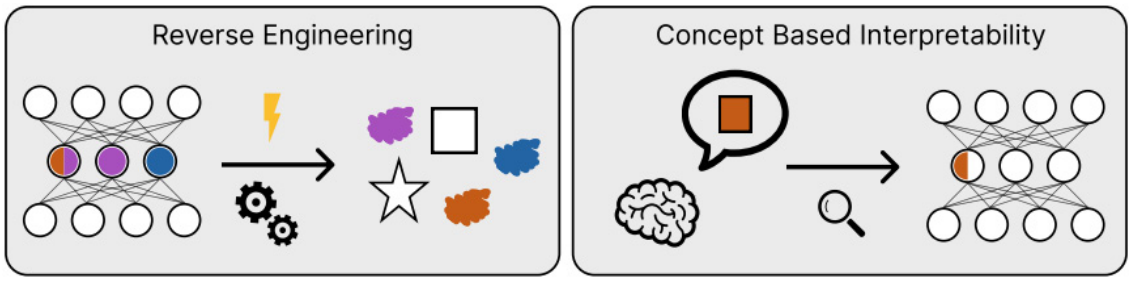


Figure 11.20 – Reverse engineering (https://arxiv.org/pdf/2501.16496)

The problem with this approach is that it is difficult to decompose neural networks into functional components. In fact, in neural networks, neurons are polysemantic and represent more than one concept. So, the interpretation of single components is not very useful, and can instead be misleading. Authors today focus on trying to decompose into functional units that incorporate multiple neurons even on multiple layers. Since these concepts are represented by multiple neurons (superimposition hypothesis), attempts are made to disentangle this sparse representation through the use of tools that force sparsity.

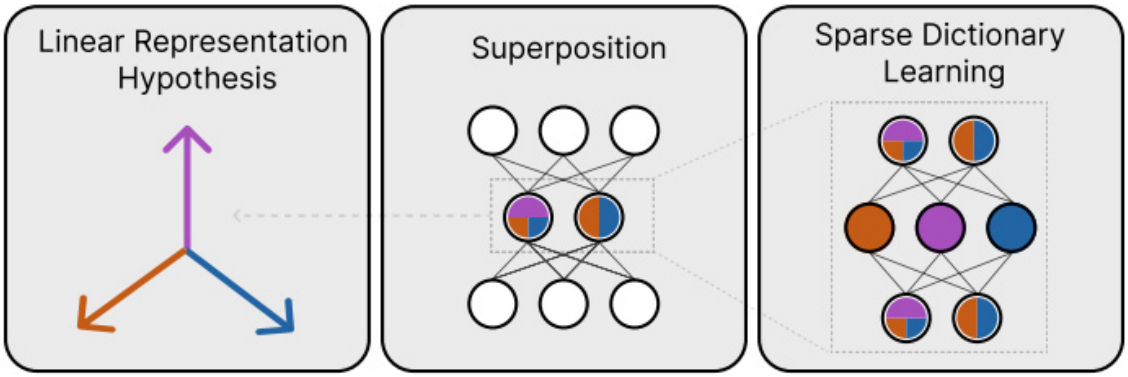


Figure 11.21 – Disentangle superimposed representation with SDL (https://arxiv.org/pdf/2501.16496)

This shift toward decomposing the model into functional units that incorporate multiple neurons and layers requires new techniques.

Sparse dictionary learning (SDL) includes a number of approaches that allow a sparse representation of what a model has learned. **Sparse autoencoders (SAEs)** are one such approach that allows us to learn sparse features that are connected to model features and make what the model has learned more accessible. SAEs use an encoder and decoder to sparsify the superimposed representation within the model.

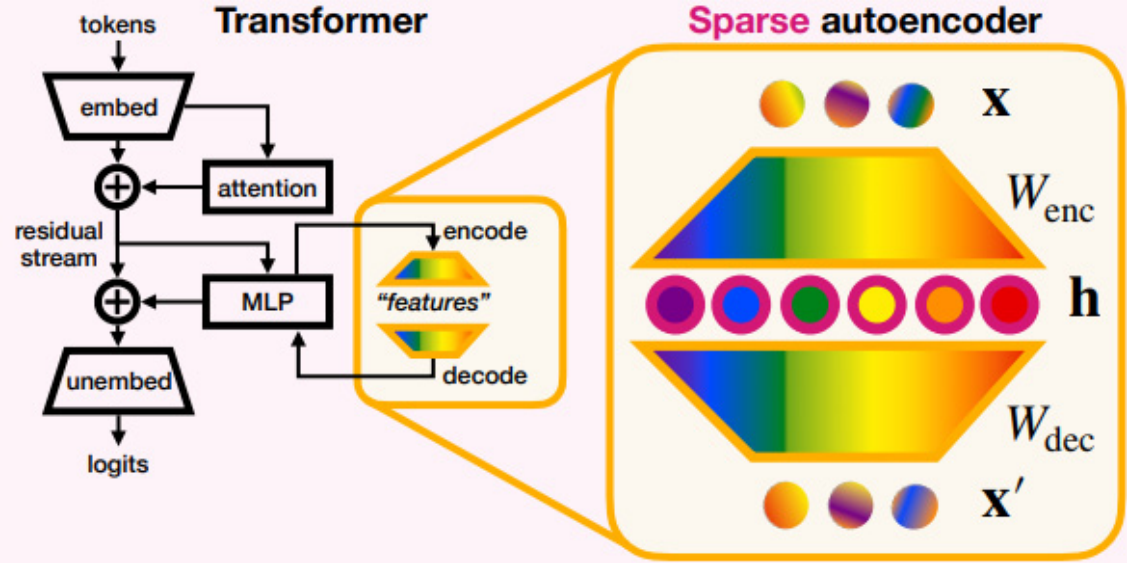


Figure 11.22 – Illustration of an SAE applied to an LLM (https://arxiv.org/pdf/2404.14082)

SAEs allow us to identify features that are human interpretable, and through sparsity, we try to learn a small number of features. At a fundamental level, SAEs can extract features related to individual words or tokens, such as word frequency features (activations that correspond to high-frequency vs. low-frequency words), and part-of-speech features (features that selectively activate for nouns, verbs, or adjectives). SAEs often capture syntactic rules embedded within LLMs, such as activations that fire for certain syntactic patterns (e.g., subject-verb-object structures) or features corresponding to syntactic dependencies, such as whether a word is a noun modifying another noun. In addition, it is also possible to identify high-level features such as neurons that fire for texts about specific domains (e.g., politics, science, or sports) and whether a sentence expresses positive, negative, or neutral sentiment. Lastly, some features can be also related to writing style and discourse structure, such as distinguishing between academic writing and casual conversation, programming languages versus human language, or distinct writing styles of certain authors (e.g., Shakespeare vs. X/Twitter posts).

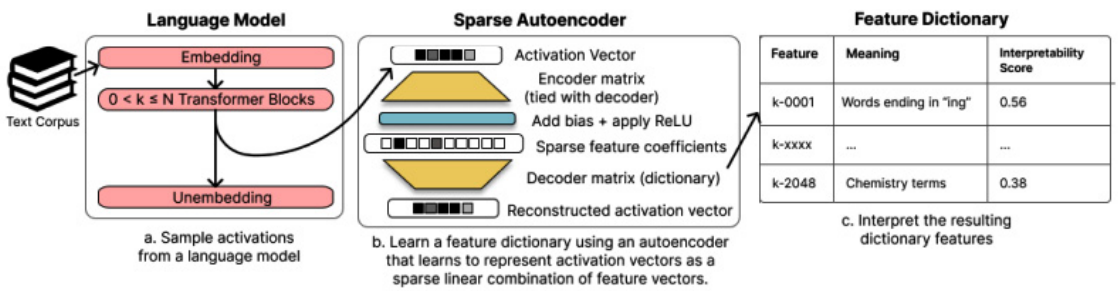


Figure 11.23 – SAE training overview (**https://arxiv.org/pdf/2309.08600**)

Some features learned by SAEs may not reflect real knowledge but rather random statistical properties of the model's embeddings. In addition, SAEs sometimes learn spurious correlations in hidden layers rather than meaningful conceptual structures. Also, SAEs focus on only one layer at a time, not considering that different neurons in different layers may interact for the same concept. Despite the associated cost, SAEs are considered a promising method for analyzing model behavior. At the same time, it was proposed to train LLMs in a more interpretable and scattered manner. The use of sparsity in the model weights aid interpretability. Techniques such as pruning and other similar techniques introduce zeros into the model weights, effectively erasing them. Sparsity eliminates connections between neurons; this makes it easier to follow the flow of information and better understand the model's decision-making process (or the relationship connecting input and output). Mixture-of-experts has a similar effect and thus makes it more interpretable.

Interpretability techniques are now critical to understanding the behavior of LLMs and preventing dangerous behaviors from emerging, such as deceiving users, showing bias, giving wrong answers especially to please users' beliefs (a phenomenon called "sycophancy"), and learning spurious correlations. As parameters and training have increased, models have become increasingly sophisticated in their responses, increasingly verbose, and persuasive, making it difficult for the user to understand whether an answer is correct. In addition, these models are now deployed with the public, which means users with malicious intentions can conduct attacks such as data poisoning, jailbreaking, adversarial attacks, and so on. Interpretability helps to monitor the behavior of the model in its interaction with the public, highlight where there have been failures, and address them in real time. Interpretability is an important requirement for model safety because it allows us not only to identify problem behaviors but also to identify which components are responsible for them. Once we have identified components associated with unintended behaviors, we can intervene with steering.

In addition, today, there is much more attention to privacy, and *machine unlearning* is the application field that deals with scrubbing the influence of particular data points on a trained machine learning model. For example, regulatory questions may require that we remove information concerning a person from our model. Machine unlearning deals with trying to remove this information without having to train the model from scratch. Machine unlearning is related to interpretability, as decomposition techniques allow us to localize concepts and information in model parameters. More generally, we want to have the ability to be able to edit model knowledge (such as correcting factual errors, removing copyrighted content, or eliminating harmful information like instructions for weapon construction). Editing requires being able to intervene on model parameters in a surgical manner without destroying additional knowledge and other capabilities. Editing is even more complex than unlearning because it means rewriting model knowledge. Interpretability techniques allow us to understand whether editing or unlearning has worked and then to monitor the process.

Interpretability is also useful in trying to predict how the model will perform in new situations, and thus avoid safety risks. Some behaviors of the model may, in fact, appear only in unanticipated situations, and may not manifest themselves when conducting a standard evaluation. For example, we can identify susceptibility or potential backdoors before these are discovered by users. Considering that today's LLMs are increasingly connected to tools, any misuse can have effects that propagate. For example, if an LLM is connected to finance databases, it could be exploited to extract information about users. Or an LLM that shops online could be exploited for fraud and buying fraudulent products. Fine-tuning and other post-training steps can lead to the emergence or exacerbation of behaviors that were not present in the pre-trained model. In addition, some properties seem to emerge at scale and are difficult to predict when we train a smaller model. Often, smaller versions of the final architecture are trained when designing a new architecture. Smaller models may not have problems that emerge only at scale.

Interpretability also has important commendable aspects; understanding the behavior of the model and its components allows us to be able to

speed up inference. For example, if some computations are unnecessary, we could turn them off, or use the knowledge gained to distill a more efficient model. In addition, we could identify components that impact either positive or negative reasoning.

Another intriguing aspect of interpretability is that it can be used for new discoveries (commonly called **microscope AI**). In other words, you can investigate a model that has been trained on certain data, and you can use interpretability techniques to gain insights into it. You can use these techniques to identify patterns that might have eluded humans. For example, after AlphaZero's success in defeating humans at chess, researchers considered extracting information from the model to identify concepts about sacrifices that humans could learn. In this paper (https://arxiv.org/abs/2310.16410), Schut et al. (2023) identified these concepts or patterns to see how the model had a different representation of the game.

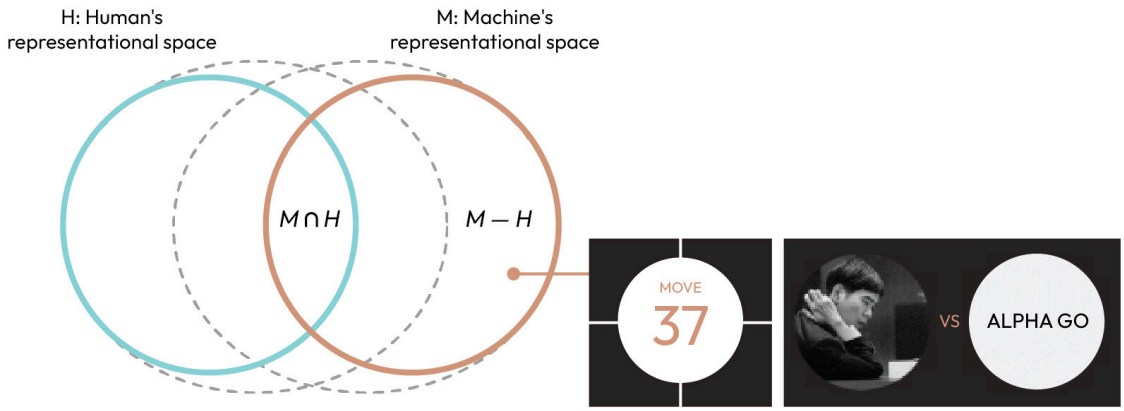


Figure 11.24 – Learning from machine-unique knowledge (https://arxiv.org/pdf/2310.16410)

LLMs have a large memory, and humans express themselves through language; this allows them to analyze and conduct hypotheses about human psychology. Interpretability then is an approach that allows us to not only better understand the model but also be able to use models as a tool to better understand humans. In the next section, we will discuss how models can potentially approach human intelligence.

The road to artificial general intelligence

"Artificial general intelligence (AGI) is a hypothesized type of highly autonomous artificial intelligence (AI) that would match or surpass human capabilities across most or all economically valuable cognitive work. It contrasts with narrow AI, which is limited to specific tasks. Artificial superintelligence (ASI), on the other hand, refers to AGI that greatly exceeds human cognitive capabilities. AGI is considered one of the definitions of strong AI."

This is the definition of **artificial general intelligence** (**AGI**) according to Wikipedia. Before imagining AI capable of surpassing humans, one must ask whether AI has caught up with human capabilities. In general, before the advent of ChatGPT, this debate did not begin (at least for the general public). This is because the previous models had superhuman capabilities only for specialized applications. For example, AlphaGo had been able to defeat human champions with relative ease, but no one thought that what makes us human was knowing how to play Go. Models such as DALL-E and ChatGPT, on the other hand, have begun to raise questions in the general audience as well. After all, generating art or creative writing are skills that are generally connected with humans. This feeling was reinforced when ChatGPT and other LLMs were able to pass university or medical and legal licensing exams.

We discussed creativity and reasoning in previous subsections. The current consensus is that LLMs do not exhibit true reasoning or creativity skills. They are sophisticated stochastic pattern machines, and their ability to find patterns in the whole of human knowledge makes them extraordinarily effective.

If LLMs are not capable of showing a level of human intelligence today, one may wonder what that might bring to AGI. Until now, it has been believed that it was possible to achieve AGI simply by scaling parameters and training. According to the idea of emergent properties, reasoning and creativity should appear at some point in the scaling (by increasing the size of the model and the number of tokens used for training, without our being able to predict it, the model should begin to show true reasoning). Today, most researchers do not believe that this is possible, nor that post-training techniques will suffice.

Moreover, scaling is not possible indefinitely. Even if we could invest enormous amounts of money and resources, there is not enough text to create models that grow linearly. In fact, humans generate a limited amount of text, and we are approaching the limit of the stock of text generated by humans.

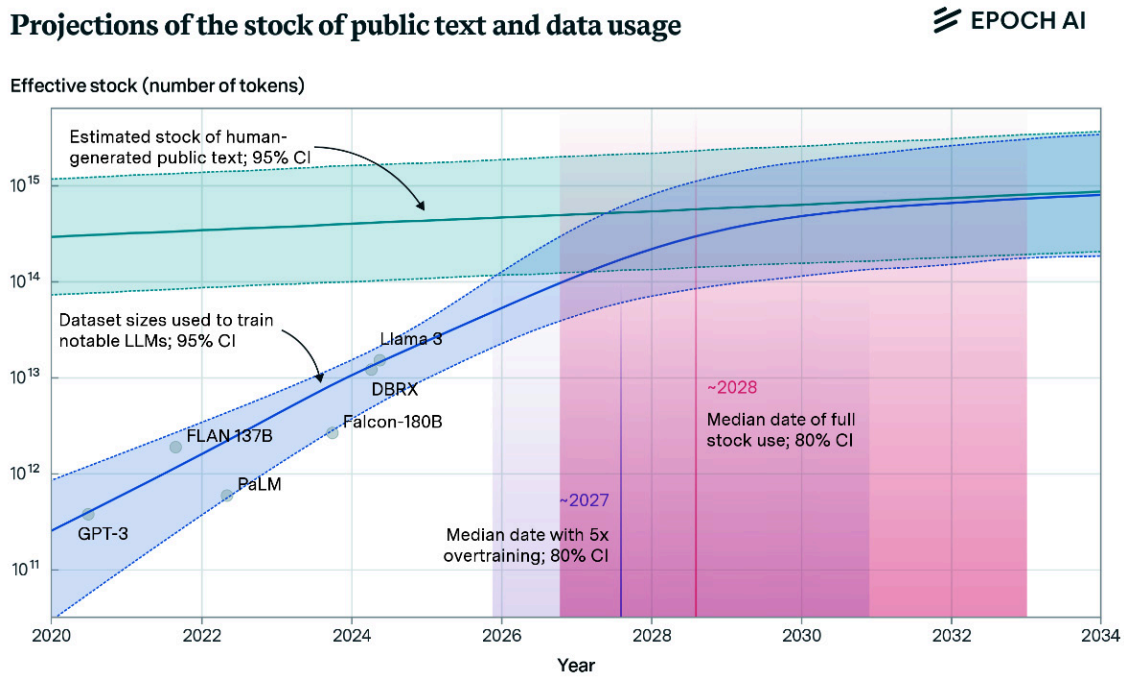


Figure 11.25 – Projections of the stock of public text and data usage (**https://epoch.ai/blog/will-we-run-out-of-data-limits-of-llm-scaling-based-on-human-generated-data**)

The solution to this could be the use of synthetic data. Synthetic data, however, can be considered a kind of “knowledge distillation” and can lead to model collapse. Models that are trained with synthetic data go into collapse, showing that performance degrades rapidly.

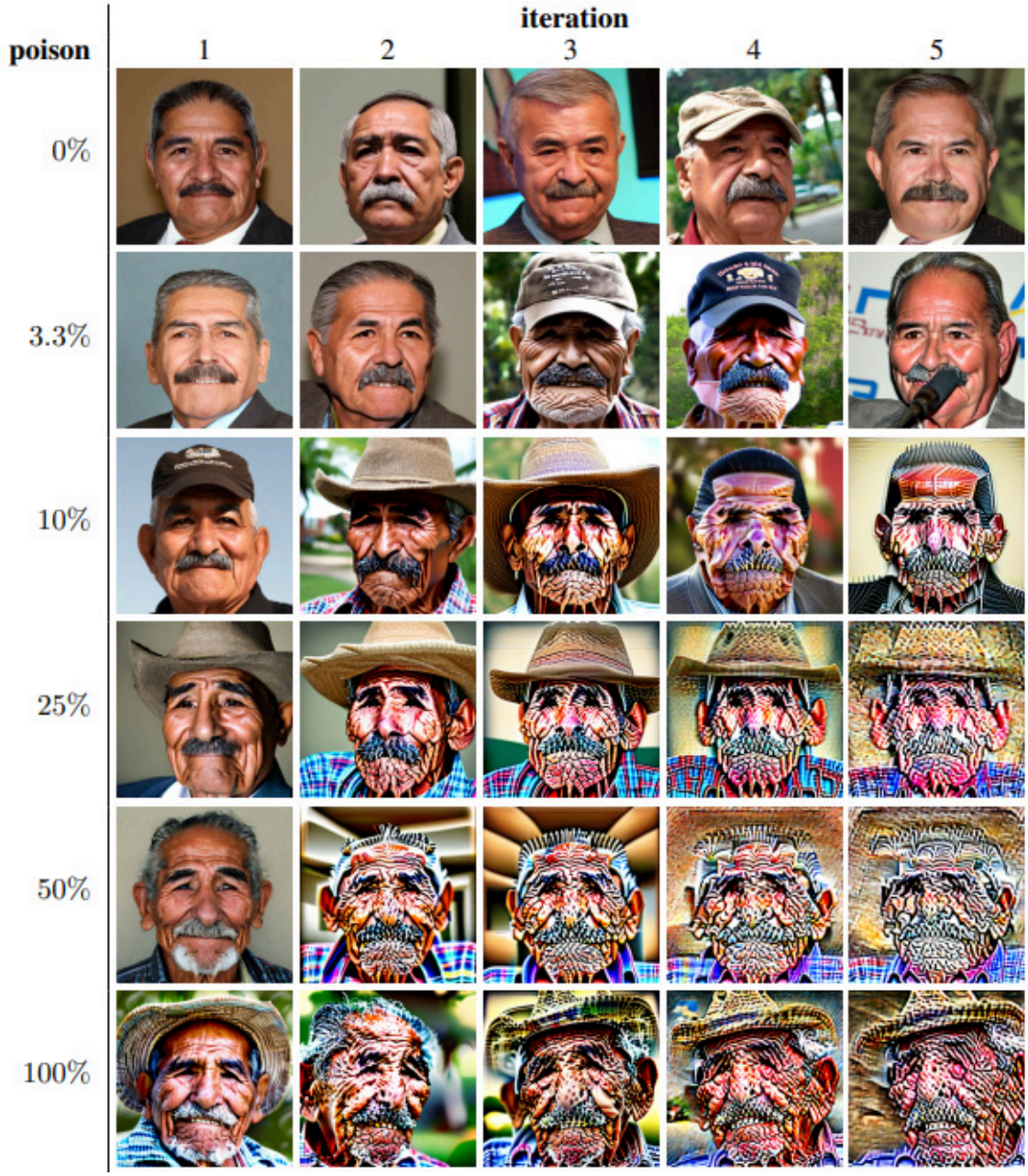


Figure 11.26 – Examples generated after iterative retraining for different compositions of the retraining dataset, from 0% synthetic data to 100 % synthetic data (**https://arxiv.org/pdf/2311.12202**)

If scaling is not the solution, some researchers propose that the key lies in developing a “world model.” That is, much like the human brain constructs an internal representation of the external environment, building such structured representations could be essential to advancing the capabilities of LLMs. This representation is used to imagine possible actions or consequences of actions. This model would also be used to generalize tasks we have learned in one domain and apply them to another. Today, some researchers suggest that LLMs have a rudimentary model of the world and that this can also be visualized. For example, Gurnee (2023)

states that LLMs form a rudimentary “world model” during training and that it shows spatiotemporal representations.

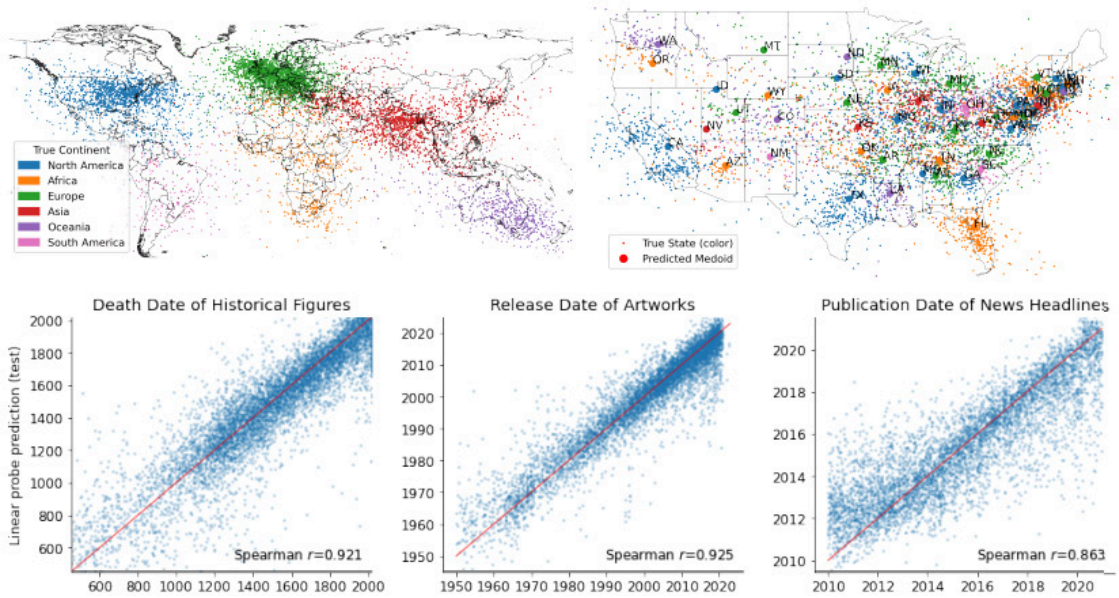


Figure 11.27 – Spatial and temporal world models of Llama-2-70b (**https://arxiv.org/pdf/2310.02207**)

These spatiotemporal representations are far from constituting a dynamic causal world model, but they seem to be the first elements for its evolution. However, there is no consensus on whether these world models can then evolve into something that is robust and reliable for conducting simulations or learning causal relationships as in humans. For example, in one study (Vafa, 2024), transformers failed to create a reliable map of New York City that can be used to conduct predictions and then used to guide.

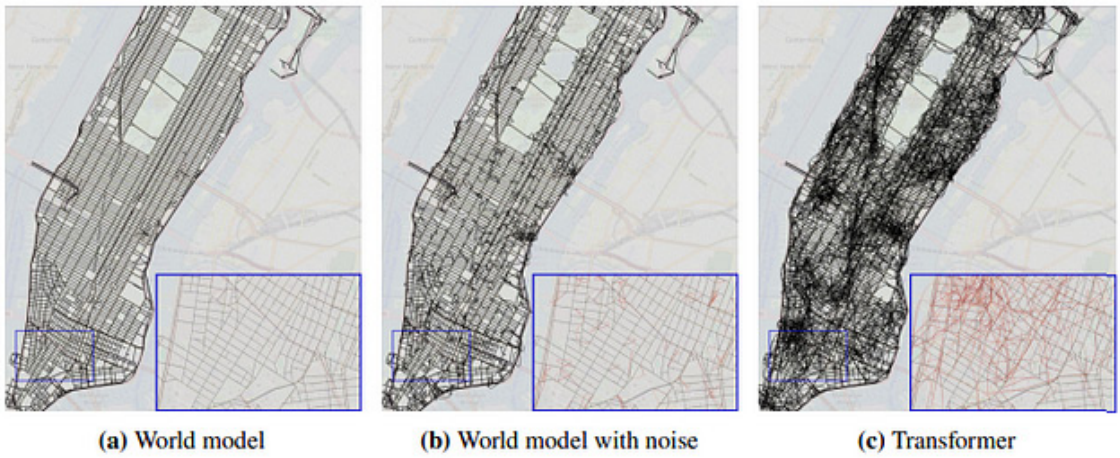


Figure 11.28 – Reconstructed maps of Manhattan from sequences produced by three models (**https://arxiv.org/pdf/2406.03689**)

Certainly, there is a wealth of information in language that can be learned, and that enables LLMs to be able to solve a large number of tasks. However, some researchers suggest that this is not enough and that models should be embodied (being used in a physical agent and being able to interact physically with the environment) in order to really make a quantum leap (including being able to learn a more robust world model). To date, this is a hypothesis and remains an open question.

Ethical questions

An article was recently published suggesting that fully autonomous AI agents should not be developed (Mitchell, 2025). While this might seem drastic, it still emphasizes the risks that autonomous agents can bring:

The development of AI agents is a critical inflection point in artificial intelligence. As history demonstrates, even well-engineered autonomous systems can make catastrophic errors from trivial causes. While increased autonomy can offer genuine benefits in specific contexts, human judgment and contextual understanding remain essential, particularly for high-stakes decisions.

The authors identify a series of levels for agents, in which humans progressively cede control of a process to the agent until the AI agent takes complete control. Recent developments in AI show how we are moving closer to creating processes where agents are in charge of an entire process.

Agentic Level	Description	Term	Example Code	Who's in Control?
☆☆☆☆	Model has no impact on program flow	Simple processor	`print_llm_output(llm_response)`	Human
★☆☆☆	Model determines basic program flow	Router	`if llm_decision(): path_a() else: path_b()`	Human: How functions are done; System: When
★★☆☆	Model determines how functions are executed	Tool call	`run_function(llm_chosen_tool, llm_chosen_args)`	Human: What functions are done; System: How
★★★☆	Model controls iteration and program continuation	Multi-step agent	`while should_continue(): execute_next_step()`	Human: What functions exist; System: Which to do, when, how
★★★★	Model creates & executes new code	Fully autonomous agent	`create_code(user_request); execute()`	System

Figure 11.29 – Levels of agents (https://arxiv.org/pdf/2502.02649)

In ***Chapter 3***, we discussed the risks associated with LLMs, whereas, in this subsection, we want to discuss in detail the risks that are associated with AI agents. Clearly, many of the risks of agent systems arise from LLMs (an LLM is central to an agent system), but extending an LLM's ability with tools creates or exacerbates new risks.

Before addressing some of the risks in detail, we would like to discuss one of the least underestimated risks of generative AI: namely, the risk of anthropomorphizing agents. As we mentioned previously, LLMs have no level of consciousness nor do they generate real emotions. LLMs emulate the distribution with which they are trained; this makes them appear as though they can emulate emotions (this clearly does not mean that they actually possess or express emotions). This must be taken into account in interactions with chatbots or other social applications in which an LLM is present. These "perceived emotions" affect not only users but also re-

searchers who must interpret the results of agents. Their ability to emulate emotions can be an effective tool for studies that simulate human behaviors, but excessive anthropomorphization risks creating misinformation and misattribution of results. In addition, anthropomorphization can lead to the risk of creating parasocial relationships between users and AI agents (a risk that will become greater when agents are embodied and thus capable of physical interaction).

Linked to the risk of anthropomorphization is the risk of excessive influence on users. There is the risk of a user being over-reliant and over-confident in an agent. Whether it is because of errors (such as hallucinations) or malicious behavior (poisoning or hacking), a user should be sufficiently skeptical of an agent's behavior. Influence risk is considered a group of risks that influence the user's behavior and beliefs:

- **Persuasion**: Refers to the ability of a model to influence a user's behavior. This can be especially problematic when an agent forces a transformative choice on the user or solicits behaviors that are harmful.
- **Manipulation**: Refers to agents that bypass an individual's rational capabilities (such as misrepresenting information or exploiting cognitive bias) to influence decision-making. This behavior could also emerge as a byproduct of poor design choices, creating a product that keeps the user engaged, or personalization for the purpose of creating trust. It is morally problematic because it does not respect the user's autonomy and could force the user into behaviors that are harmful to himself.
- **Deception**: Refers to strategies that cause an individual to form a false belief. This is likely to push a user toward behaviors that could be harmful to themselves because they are confused by false beliefs.
- **Coercion**: Implies an individual choosing something because they have no other acceptable alternative. This risk can be physical (with embodied agents) or psychological (also chatbots).
- **Exploitation**: Implies taking unfair advantage of an individual's circumstances. AI agents can be programmed to be exploitative (we can imagine an AI agent in a casino trying to push users to spend as much as possible).

These behaviors can be exploited by malicious actors. For example, an agent's persuasion skills can be used for the spread of misinformation online. LLMs are capable of generating impressive amounts of text that can seem authoritative. An LLM in itself can generate hallucinations, but it can be used for the purpose of intentionally generating fake news with a specific purpose. The use of agents allows an LLM to use additional tools (generate images and videos and retrieve information) and feed them directly into communication channels. Paradoxically, since this fake news is difficult to intercept by humans, agents can also be used to combat the spread of AI-generated misinformation. Agents can then be used to generate disinformation at scale, with the cost to generate gradually dropping (and is still lower than employing humans). In addition, agents make it possible to search for information about the victim and thus generate customized content to be more effective.

Misinformation can be used to reinforce bias toward individuals or groups. This content (text, images, audio, and video) can be used to influence political elections or drive citizen outrage. In addition, apart from disinformation, agents can also produce other types of harmful content, including depictions of nudity, hate, or violence.

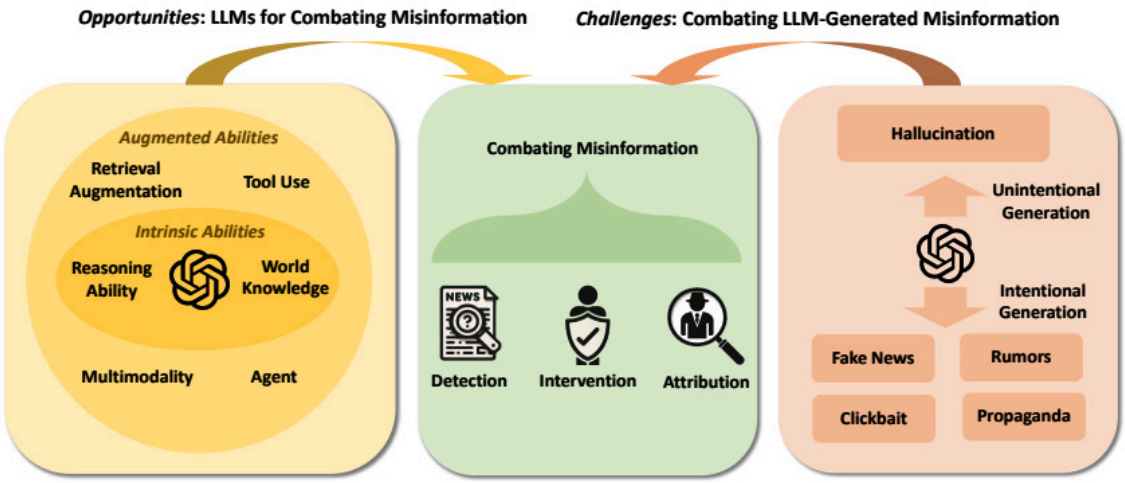


Figure 11.30 – Opportunities and challenges of combating misinformation in the age of LLMs (https://arxiv.org/pdf/2311.05656)

Malicious actors can also use agents for additional purposes such as phishing attacks, cyberattacks, or scams. In fact, LLMs can also generate harmful code that can be used to steal money or information. Chatbots can be used for the purpose of gaining trust and convincing someone to share information or earnings. There are also devious ways to attack an

agent; for example, we can imagine an agent who conducts purchases for a user can be infiltrated by a bad actor who poisons the agent and prompts it to conduct fraudulent purchases. The planned deployment of AI assistants in fields such as healthcare, law, education, and science multiplies the risks and severity of possible harm. In addition, many of the AI agents today are natively multimodal (multiple possible types of inputs and outputs) and leverage deep reasoning models that are capable of more reasoning and planning. In addition, unlike LLMs, they also incorporate memory systems, all of which add risk. For example, an agent tasked with conducting a cyberattack could retrieve from memory successful past attacks or discard outdated techniques, search for information on online vulnerabilities, generate and execute code, and devise a multi-step strategy. As has been seen, a malicious actor can interfere with an LLM in several ways, for example, through prompt injection or information extraction. An LLM acquires sensitive information during its training that can be extracted. Agents can be connected to sensitive databases, and there are techniques to make LLMs extract the content. In addition, agent misuse can be conducted by authoritarian governments. For example, governments may use agents to generate misinformation or censorship and may use them for surveillance, tracking, and silencing dissent. Advanced agent systems can extract data from cell phones, cars, the Internet of Things, and more, making it easier to control the population.

Another risk is the economic impact of these agents. AI is expected to impact several aspects of the economy in terms of productivity but also in terms of employment, job quality, and inequality. The use of agents and AI in general have different associated risks:

- **Employment**: Various research estimates that 47 percent of jobs are at risk of automation, especially jobs that are characterized by routines and physical tasks such as driving and manufacturing. Advances in LLMs have also brought alarm to jobs that involve generating and manipulating information, and that are normally associated with higher levels of education such as translators, tax advisers, or even software engineers. AI could therefore accelerate job loss for positions requir-

ing skilled labor, without creating a number of positions with which to absorb displaced jobs.

- **Job quality**: Some initial studies have suggested that the use of AI could make workers more productive and increase the wages of workers. Some studies, however, place emphasis on the possibility that employers may more efficiently monitor their employees with greater stress. Other studies note how the introduction of robots may reduce the physical workload in manufacturing but push workers to work faster, with less human contact and more supervision.
- **Inequality**: On the one hand, technological development has decreased inequality between different countries. At the same time, intra-country income inequality has increased, with a sharper separation of wealth between the richest and poorest. There are few studies looking at how AI may impact inequality, but some studies suggest that firms are best able to draw on AI with increased productivity and earnings, while workers are at risk of displacement and thus reduced income. Some studies suggest that high-income occupations may benefit from using AI, while others will be impacted. For example, AI assistants appear to impact junior positions by reducing them. In addition, most of the leading AI research labs, start-ups, and enterprises are located in certain geographic areas, with the risk of concentration of well-paying positions. Conversely, AI also creates low-paying jobs, especially in data creation and data acquisition.

To date, AI tools are not sophisticated enough to replace humans, but some effects on employment are already visible. Tools such as ChatGPT and DALL-E have already had an impact on the “creative economies” (which includes writers, artists, designers, photographers, content creators, and so on) with reduced positions and earnings.

Another risk is the environmental impact of generative AI. Data and compute underlie the training and use of AI systems. Thus, hardware and infrastructure for storage and processing (including data centers and telecommunications) are required for the creation and use of agents. Creating the necessary hardware has an environmental impact (mining of rare earths, energy to build and ship them, water in plants, and use of chemicals). Then, a model requires energy to be drawn, built, and de-

ployed (operational costs). Beyond training, deployment in inference also requires resources and energy consumption. Energy consumption and corresponding carbon dioxide emissions associated with LLM training are increasing over time. As can be seen in the following figure, carbon dioxide production linearly increases with energy consumption for training (directly related to the number of parameters and the increase in training):

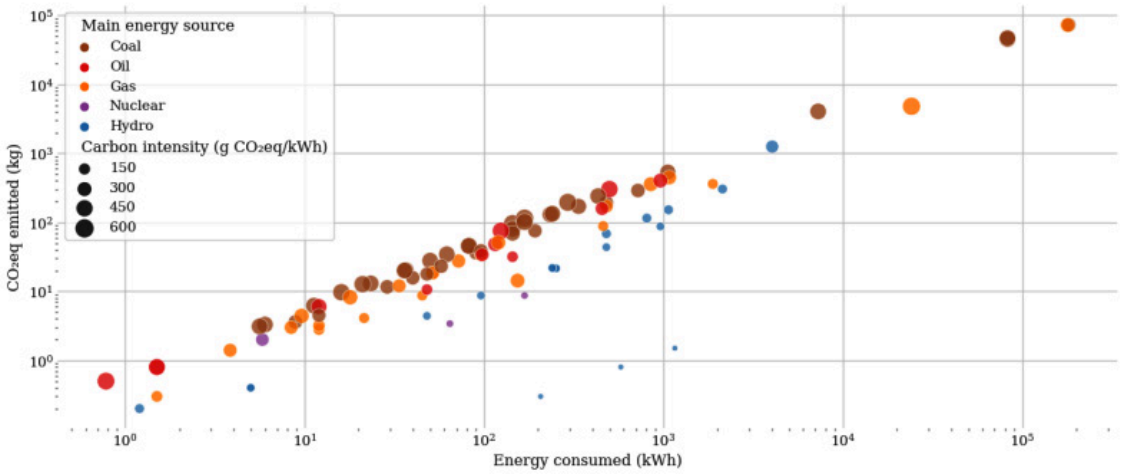


Figure 11.31 – Estimated energy consumed (kWh) and CO2 (kg) by different models (https://arxiv.org/pdf/2302.08476)

Considering the increase in users who use LLMs (or services that include LLMs) on a daily basis, the impact of training on emissions is only a fraction. Today, inference is supposed to be increasingly important (it has been estimated that 60% of machine learning energy use at Google from 2019–2021 was attributable to inference).

These are some of the possible risks with LLMs and agents. To date, strategies are being studied to try to address and mitigate these risks.

Summary

This chapter presented how some industries will be revolutionized by agents. The AI revolution goes beyond these industries and will have a large-scale impact. This book, however, provided a serious and structured introduction to the technical component that will drive this revolution, giving you the tools to understand the future that will come (and is already upon us). Apart from the sense of wonder that this technological

revolution may inspire, we wanted to remind you that there are still technical and ethical challenges that should not be overlooked.

This chapter closes this book but leaves open a series of questions and challenges for the future. Readers who have followed us up to this point can find in this final chapter suggestions for leveraging what they have learned at the industry level and at the research level.

Further reading

- Luo, *BioGPT: Generative Pre-trained Transformer for Biomedical Text Generation and Mining*, 2022, **https://academic.oup.com/bib/article/23/6/bbac409/6713511**
- Yao, *Health System-scale Language Models are All-purpose Prediction Engines*, 2023, **https://www.nature.com/articles/s41586-023-06160-y**
- Singhal, *Towards Exper-level Medical Question Answering with Large Language Models*, 2023, **https://arxiv.org/abs/2305.09617**
- Gao, *Empowering Biomedical Discovery with AI Agents,* 2024, **https://www.cell.com/cell/fulltext/S0092-8674(24)01070-5**
- Gu, *A Survey on LLM-as-a-Judge*, 2024, **https://arxiv.org/abs/2411.15594**
- Ning, *A Survey of WebAgents: Towards Next-Generation AI Agents for Web Automation with Large Foundation Models*, 2025, **https://arxiv.org/abs/2503.23350**
- Xu, *A Survey on Robotics with Foundation Models: Toward Embodied AI*, 2024, **https://arxiv.org/pdf/2402.02385**
- Hu, *A Survey on Large Language Model Based Game Agents*, 2025, **https://arxiv.org/pdf/2404.02039**
- Bousateouane, *Physical AI Agents: Integrating Cognitive Intelligence with Real-World Action*, 2025, **https://arxiv.org/pdf/2501.08944v1**
- Zeng, *Large Language Models for Robotics: A Survey*, 2023, **https://arxiv.org/pdf/2311.07226**
- Bansal, *Challenges in Human-Agent Communication*, 2024, **https://www.microsoft.com/en-us/research/uploads/prod/2024/12/HCAI_Agents.pdf**
- Raieli, *The Savant Syndrome: Is Pattern Recognition Equivalent to Intelligence?*, 2024, **https://medium.com/towards-data-science/the-**

savant-syndrome-is-pattern-recognition-equivalent-to-intelligence-242aab928152

- Lu, *Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity*, 2022, https://aclanthology.org/2022.acl-long.556/
- Zhao, *Calibrate Before Use: Improving Few-shot Performance of Language Models*, 2021, https://proceedings.mlr.press/v139/zhao21c.html
- Raieli, *Emergent Abilities in AI: Are We Chasing a Myth?*, 2023, https://towardsdatascience.com/emergent-abilities-in-ai-are-we-chasing-a-myth-fead754a1bf9
- Raieli, *A Focus on Emergent Properties in Artificial Intelligence*, 2025, https://github.com/SalvatoreRa/artificial-intelligence-articles/blob/main/articles/emergent_properties.md
- Xu, *Towards Large Reasoning Models: A Survey of Reinforced Reasoning with Large Language Models*, 2025, https://arxiv.org/abs/2501.09686v3
- Sprague, *To CoT or Not to CoT? Chain-of-thought Helps Mainly on Math and Symbolic Reasoning*, 2024, https://arxiv.org/pdf/2409.12183
- Raieli, *To CoT or Not to CoT: Do LLMs Really Need Chain-of-Thought?*, 2024, https://levelup.gitconnected.com/to-cot-or-not-to-cot-do-llms-really-need-chain-of-thought-5a59698c90bb
- Mirzadeh, *GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models*, 2024, https://arxiv.org/pdf/2410.05229
- Sharkey, *Open Problems in Mechanistic Interpretability*, 2025, https://arxiv.org/abs/2501.16496
- Bereska, *Mechanistic Interpretability for AI Safety -- A Review*, 2025, https://arxiv.org/abs/2404.14082
- Cemri, *Why Do Multi-Agent LLM Systems Fail?*, 2025, https://arxiv.org/pdf/2503.13657
- Raieli, *Creativity in LLMs: Optimizing for Diversity and Uniqueness*, 2025, https://medium.com/data-science-collective/creativity-in-llms-optimizing-for-diversity-and-uniqueness-f5c7208f4d99
- Boden, *The Creative Mind*, 2003, https://www.routledge.com/The-Creative-Mind-Myths-and-Mechanisms/Boden/p/book/9780415314534

- Peeperkorn, *Is Temperature the Creativity Parameter of Large Language Models?*, 2024, **https://arxiv.org/abs/2405.00492**
- Benedek, *To Create or to Recall? Neural Mechanisms Underlying the Generation of Creative New Ideas*, 2014, **https://www.sciencedirect.com/science/article/pii/S1053811913011130**
- Raieli, *You're Not a Writer, ChatGPT — But You Sound Like One*, 2024, **https://levelup.gitconnected.com/youre-not-a-writer-chatgpt-but-you-sound-like-one-75fa329ac3a9**
- Raieli, *How Far Is AI from Human Intelligence?*, 2025, **https://levelup.gitconnected.com/how-far-is-ai-from-human-intelligence-6ab4b2a5ce1c**
- Villalobos, *Will We Run Out of Data? Limits of LLM Scaling Based on Human-generated Data*, 2022, **https://arxiv.org/abs/2211.04325**
- Feng, *How Far Are We From AGI: Are LLMs All We Need?*, 2024, **https://arxiv.org/abs/2405.10313**
- Karvonen, *Emergent World Models and Latent Variable Estimation in Chess-Playing Language Models*, 2024, **https://arxiv.org/pdf/2403.15498v2**
- Li, *Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task*, 2022, **https://arxiv.org/abs/2210.13382**
- Bowman, *Eight Things to Know about Large Language Models*, 2023, **https://arxiv.org/pdf/2304.00612**
- Shumailov, *AI Models Collapse When Trained on Recursively Generated Data*, 2024, **https://www.nature.com/articles/s41586-024-07566-y**
- LessWrong, *Embodiment is Indispensable for AGI*, 2022, **https://www.lesswrong.com/posts/vBBxKBWn4zRXwivxC/embodiment-is-indispensable-for-agi**
- Tan, *The Path to AGI Goes through Embodiment*, 2023, **https://ojs.aaai.org/index.php/AAAI-SS/article/view/27485**
- Mitchell, *Fully Autonomous AI Agents Should Not be Developed*, 2025, **https://arxiv.org/pdf/2502.02649**
- Diamond, *On the Ethical Considerations of Generative Agents*, 2024, **https://arxiv.org/abs/2411.19211**
- Siqueira de Cerqueira, *Can We Trust AI Agents? An Experimental Study Towards Trustworthy LLM-Based Multi-Agent Systems for AI Ethics*, 2024, **https://arxiv.org/abs/2411.08881**

- Chaffer, *Decentralized Governance of Autonomous AI Agents*, 2024, **https://arxiv.org/abs/2412.17114v3**
- Gabriel, *The Ethics of Advanced AI Assistants*, 2024, **https://arxiv.org/pdf/2404.16244**
- Chen, *Combating Misinformation in the Age of LLMs: Opportunities and Challenges*, 2023, **https://arxiv.org/abs/2311.05656**
- Luccioni, *Counting Carbon: A Survey of Factors Influencing the Emissions of Machine Learning*, 2023, **https://arxiv.org/abs/2302.08476**

Table of contents collapsed